One operator, three decisions, one evidence trail

AI-generated concept illustration; not an installed control room.

Voltage support
Which inverter command meets the limits?

Restoration
Which permitted path can restore service?

Cyber response
Which evidence supports the estimated state?

Learning goal: follow a request through tools, decisions and checks until an operator can explain why to accept, hold or reject it.

U.S. DOE, ADMS Insights (2015). Illustration: `figures/AI_Illustration_Provenance.md`.

Next: What will we build in three hours?

A three-hour route from diagrams to operator review

Minutes	Explain	Do
0–40	Graphs, state, decisions and agents	Trace a LangGraph toy; reject stale evidence
40–82	Voltage and MPC/SAC proposals	Compare controls and Llama trials
82–92	**Break**	Keep the notebook kernel ready
92–119	Restoration and discrete choices	Change tie capacity and inspect service
119–149	Cyber response and estimation	Challenge corrupted measurements
149–162	Research and scientific comparison	Compare outcomes, evidence and latency
162–180	Control-room capstone	Explain an acknowledgment or a hold

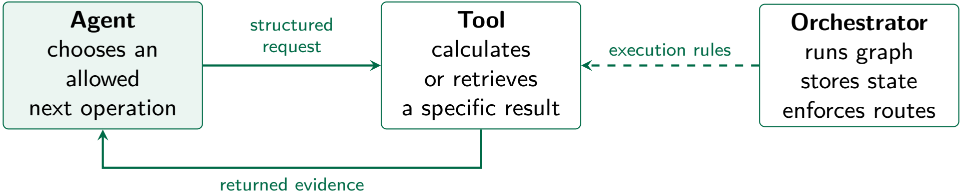
Each exercise: predict → run → explain the returned evidence.

Labels matter: fresh calculation, saved Llama response and live call are different experiments.

Workshop route: this tutorial. Evidence evaluation: PowerAgentBench-SS (2026 preprint).

Next: Which part of the software makes a decision?

Tool, agent and orchestrator have different jobs



Agent
chooses an
allowed
next operation

structured
request

Tool
calculates
or retrieves
a specific result

execution rules

Orchestrator
runs graph
stores state
enforces routes

returned evidence

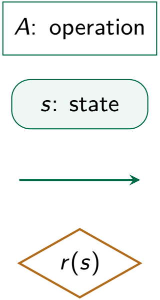
Example: Llama proposes a trial; a power-flow tool returns voltages; the orchestrator requires verification before a recommendation.

An LLM call can live inside a graph node. A numerical node can run without any LLM. Several roles can use the same Llama backbone.

Yao et al., ReAct (ICLR 2023); LangGraph workflows and agents, accessed 2026-10-17.

Next: How do we draw these responsibilities clearly?

Read the diagram: operation, state, dependency and decision



A function that consumes data and returns a result.

The current record: request, command, results and status.

$A \rightarrow B$: finish the required result from A before B.

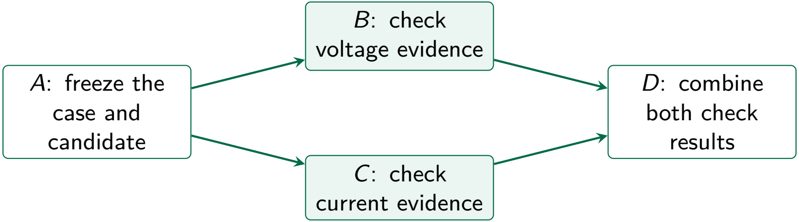
r: routing rule; its returned label selects the next path.

A workflow graph shows **computation dependencies**. Its edges are not the feeder's electrical lines.

LangGraph Graph API; NetworkX topological sort. Tutorial schematics.

Next: Which operations could run independently?

A dependency DAG can split work and join the evidence


B: check voltage evidence
A: freeze the case and candidate
D: combine both check results
C: check current evidence

DAG = directed acyclic graph: arrows have direction; no path returns to its starting node.

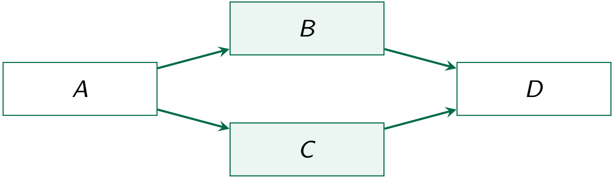
$G = (N, E)$: $N = \{A, B, C, D\}$ is the node set; E is the set of dependency arrows.

Fan-out: B and C receive the same frozen input. **Join:** D needs both outputs.

NetworkX DAG ordering; LangGraph joins. Conceptual checks; not the G-lab topology.

Next: Which execution orders preserve every dependency?

Topological order puts every prerequisite before its consumer



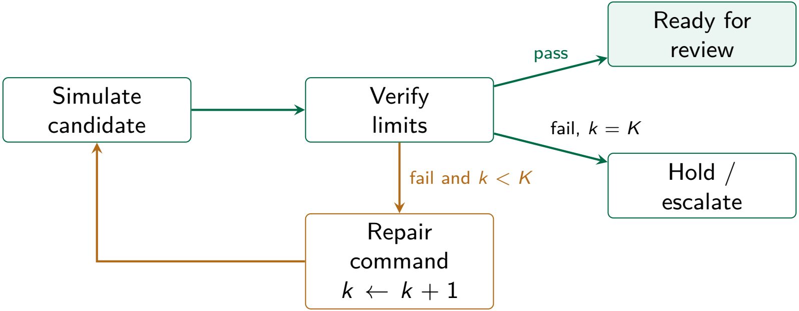
A: freeze input
B: voltage check
C: current check
D: combine

Candidate order	Does it respect all arrows?
A, B, C, D	✓ Yes.
A, C, B, D	✓ Yes; B and C are independent.
A, B, D, C	— No; D is missing C's result.

A schedule can group $\{B, C\}$ together. The join must still wait for both.

NetworkX topological sort. Tutorial's four-node example.

A retry loop is a graph with a cycle, not a DAG


Simulate candidate
Verify limits
pass
Ready for review
fail, $k = K$
Hold / escalate
fail and $k < K$
Repair command $k \leftarrow k + 1$

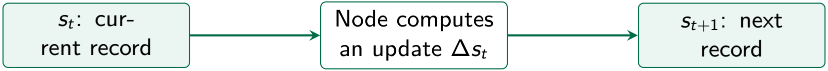
k: completed repairs; K: maximum allowed repairs; both non-negative integers.

Every changed candidate returns to simulation. A finite budget guarantees a stop *if each tool call returns*; it does not guarantee a feasible action.

LangGraph overview; Graph API examples. Retry rule is the tutorial's policy.

Typed state is the record carried through the graph

Field	Type / example	Why the next node needs it
`command`	Number, 0.75	Exact proposed input
`current`, `limit`	Numbers, same declared units	A meaningful comparison
`attempts`, `max_repairs`	Integers, 0 and 2	Remaining retry budget
`passed`	Boolean	Numerical gate result
`trace`	List of event strings	Inspectable execution history



s_t: current record

Node computes an update Δs_t

s_{t+1}: next record

t: workflow step; Δs_t: returned fields, not arithmetic subtraction.

LangGraph state; Python TypedDict. Illustrative G-lab fields.

Next: Should a new result replace or append to a field?

A reducer states how a field absorbs new information

Replace a current result

Existing `current`	1.00
New `current`	0.80
Stored result	0.80

Only the latest result belongs to the current command.

Append an event to history

`[simulate]` + `[verify]`

`[simulate, verify]`

The list-addition reducer preserves both events.

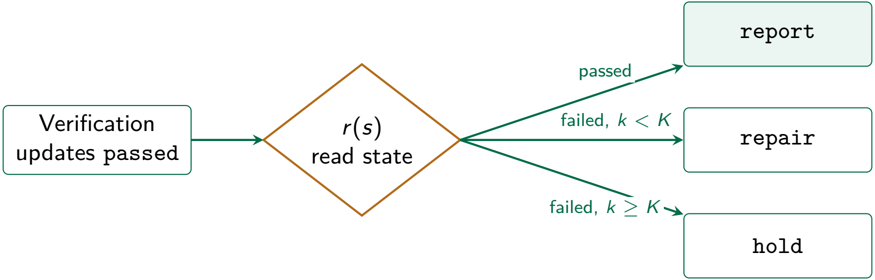
$R_k(a, b)$: reducer for field k, combining stored value a and returned update b.

Design question: if two parallel branches write the same field, what merge rule is valid? Never let accidental write order define the answer.

LangGraph reducer definitions; worked examples. Values are illustrative.

Next: How does the record choose the next edge?

Conditional routing makes the next action explicit


Verification updates `passed`
$r(s)$ read state
passed
failed, $k < K$
failed, $k \geq K$
`report`
`repair`
`hold`

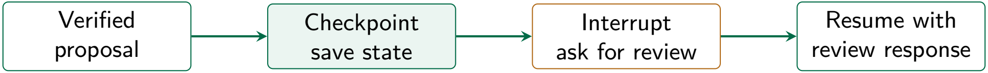
s: state; $r(s)$: one route label; k: repairs used; K: repair budget.

Separate responsibilities: verification computes admissibility; routing decides what the workflow does with that result.

LangGraph conditional-edge examples. Three-way policy: `Graph_Workflow_Lab.ipynb`, G2.

Next: Where can a human pause this process?

A checkpoint and an interrupt make review resumable

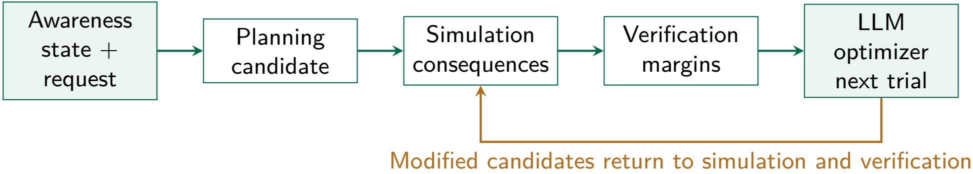

Verified proposal
Checkpoint save state
Interrupt ask for review
Resume with review response

Term	Meaning for this example
Checkpoint	Saved state, associated with a `thread_id`.
Interrupt	Pause that exposes a review request to the caller.
Resume value	Explicit response returned to the paused node.

The review node can run again on resume. Put no uncontrolled equipment side effect before the pause. The toy records acknowledgment only.

LangGraph persistence; interrupt/resume documentation, accessed 2026-10-17.

Five proposed stages share one candidate and its evidence



Awareness state + request → Planning candidate → Simulation consequences → Verification margins → LLM optimizer next trial

Modified candidates return to simulation and verification

Shared record: case, state version, command, model scope, limits, tool result and verdict.

Operator handoff: selected action, supported public explanation, alternatives and unresolved assumptions.

An agent is a responsibility; numerical functions can implement much of it. Execution and acknowledgment support the five stages.

SmartGridComm 2026 tutorial proposal, pp. 1–2. Roles: Wu et al., AutoGen (2023 preprint).

Typed, current and feasible are three separate claims

Check	Question	What a pass does not establish
Schema	Are required fields and values valid?	Correct interpretation of operator intent
Binding	Is this result for this request and command?	Authenticity of its source
Engineering gate	Does it meet modeled constraints?	Adequate model or field authorization

Illustrative result: limit 100 A; calculated current 112 A; solver converged.

Decision: REJECT. Two favorable LLM responses cannot change the declared 100 A limit.

PowerDAG (2026 preprint); Grid-Agent (2025 preprint, v3). Current example is illustrative.

Recent frameworks provide mechanisms we can test

Work	Mechanism	Classroom question
PowerAgent / PowerMCP	Engineering-tool interfaces	Can we inspect arguments and units?
PowerDAG	Retrieval and prerequisite guards	Does calculation use current state?
Grid-Agent v3	Sandbox, validation and rollback	Can rejection replace the incumbent?
X-GridAgent	Planning–coordination–action hierarchy	What must survive each handoff?
PowerAgentBench-SS	Matched tools; hidden revalidation	Is the reported claim supported?

Research lesson: compare mechanisms under stated scope. Five roles, tool use and verification already have close prior art.

PowerAgent; PowerDAG; Grid-Agent; X-GridAgent; PowerAgentBench-SS. Review: 2026-10-17.

A capability tick describes scope, not superiority

Work	Tools	Feedback	Checks	Review	Qualification
PowerAgent ecosystem	✓	✓	△	✓	Workflow-specific capabilities
PowerDAG	✓	✓	△	NR	Guards cover selected tools
Grid-Agent	✓	✓	✓	△	Sandbox; proposed oversight
X-GridAgent	✓	✓	△	△	Solver checks; prompt refinement
PowerAgentBench-SS	✓	✓	✓	△	DC benchmark; hidden revalidation
Tutorial workflow	✓	✓	✓	✓	Declared models; local readback

✓ documented △ qualified / proposed NR: not reported in inspected source.

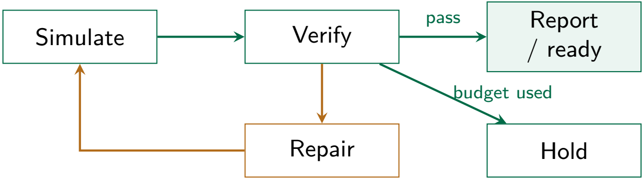
Test the benefit: same tools, scenario and trial budget; compare outcome, evidence quality and latency.

Definitions: `appendix/Power_Frameworks_Comparison.md`. Evaluation: PowerAgentBench-SS. Current versions checked 2026-10-17.

Next: Can we run the graph mechanics without an LLM?

The G lab makes every state transition visible

Notebook G0–G5 `Graph_Workflow_Lab.ipynb`: a real LangGraph toy demonstrator.



Toy response: $i = 0.4 + 0.8u$. u: dimensionless command; i: normalized illustrative current. Pass if $i \leq L$, where L is the declared limit.

Repair: reduce u by 0.25, bounded below by zero. This is *not* a feeder model, MPC or learned policy.

`Graph_Workflow_Lab.ipynb`, G1–G4; LangGraph official documentation.

G1: give the graph a record and an event history

Notebook G1 Inspect the complete state schema and the trace reducer.

```
class TrialState(TypedDict):
    command: float
    limit: float
    attempts: int
    max_repairs: int
    current: float
    passed: bool
    status: str
    acknowledged: bool
    trace: Annotated[list[str], add]
```

Read the code: `TypedDict` names fields; `Annotated` attaches the reducer; `add` appends event lists.

Predict: after simulate and verify both append one event, should `trace` contain one entry or two? **Two.**

`Graph_Workflow_Lab.ipynb`, G1; Python TypedDict; LangGraph reducers.

G2–G3: bind node functions to permitted routes

Notebook G2–G3 Read the route, then build and compile the graph.

Decision rule

```
def route(s):
    if s["passed"]:
        return "report"
    return "repair" if s["attempts"] < s["
     max_repairs"] else "hold"
```

Graph connections

```
    builder.add_edge(START, "simulate")
    builder.add_edge("simulate", "verify")
    builder.add_conditional_edges("verify",
     route,
        {"report": "report", "repair": "repair",
      "hold": "hold"})
    builder.add_edge("repair", "simulate")
    builder.add_edge("report", END)
    builder.add_edge("hold", END)
    return builder.compile(checkpointer=
     checkpointer)
```

Code to diagram: node names identify work; edges identify allowed successors; the conditional edge reads a route label.

`Graph_Workflow_Lab.ipynb`, G2–G3; LangGraph construction examples.

G4: predict three paths before running the graph

Trial	Initial u	Limit L	Repairs K	Expected path
Already admissible	0.25	0.80	2	Ready; $i = 0.60$
One repair suffices	0.75	0.80	2	Repair once; ready at $i = 0.80$
Budget exhausted	0.90	0.50	1	Repair once; hold at $i = 0.92$

u: command; L: limit; K: repair budget; $i = 0.4 + 0.8u$: toy normalized current.

Your turn: 2 minutes

Run G4 unchanged. Follow its trace. In a separate scratch cell, invoke the graph with a new limit or repair budget; predict the route first.

Actual labels: `TOY_READY` / `TOY_HOLD`. The direct-Python baseline produces the same three outcomes.

Exact toy and direct-Python comparison: `Graph_Workflow_Lab.ipynb`, G4. Values follow the declared response law and repair rule.

G5: pause, inspect the checkpoint, then resume review

Notebook G5 Use a checkpointer and a real interrupt; record no equipment action.

```
review_app = build_graph(human_review, InMemorySaver())
config = {"configurable": {"thread_id": "student-" + uuid.uuid4().hex}, "recursion_limit": 40}
pause = review_app.invoke(initial_state(0.75, 0.80, 2), config)
snapshot = review_app.get_state(config)
print("Pending node:", snapshot.next, "| acknowledged:", snapshot.values["acknowledged"])
print("Interrupt payload:", pause["__interrupt__"][0].value)
assert snapshot.next == ("report",) and not snapshot.values["acknowledged"]
response = {"approve": True, "recommendation_id": recommendation_id(snapshot.values)}
resumed = review_app.invoke(Command(resume=response), config)
```

Before resume: pending node `report`; acknowledgment is false.

After the scripted readback: inspect `TOY_ACKNOWLEDGED`; retain the same `thread_id`.

Try a rejection: it must remain distinguishable from approval. This toy does not authenticate an operator.

`Graph_Workflow_Lab.ipynb`, G5; LangGraph interrupts; persistence.

Next: Can saved evidence survive a changed request?

L1: a changed request needs fresh evidence

Notebook L0–L1 Open `Classroom_Labs.ipynb`; run setup, then the binding experiment.

```
request = lab.validate_request(dict(task=’voltage’, backend=’lindistflow3’,
    load_scale=1.0, rating_fraction=0.80, revision=1))
candidate = {’control’: [0.25, 0.0]}
evidence = lab.bind_evidence(request, candidate, {’example_only’: True, ’passed’: True})
```

Compare the saved record with	Expected binding result
Same request and command	True
Changed load with old evidence	False
Changed command with old evidence	False

Explain: hashes bind request, candidate and result. The payload is `example_only`; matching records do not prove physics or source authenticity.

`Classroom_Labs.ipynb`, L1; PowerDAG (2026 preprint).

Next: Which method should propose the real command?

Model-based planning and model-free learning supply proposals

Model-based: optimize with a model

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \ell(x_k, u_k)$$

subject to predicted behavior and operating limits.

H: horizon; k: predicted step; x_k: state; u_k: action; ℓ: declared cost.

Voltage example: MPC plans several actions and uses the first.

Model-free: learn an action rule

$$u_t = \pi_\theta(o_t)$$

from interactions and rewards during training.

t: decision time; o_t: observation; π_θ: policy with learned parameters θ.

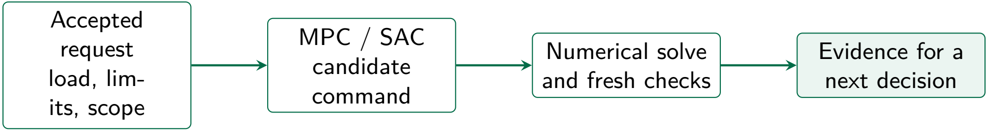
Voltage example: SAC produces continuous controls from saved weights. PPO is another option.

Shared workflow: simulate both proposals and check the same limits. Llama may select permitted follow-up work.

Rawlings, Mayne and Diehl, MPC (2017); Haarnoja et al., SAC (2018); Schulman et al., PPO (2017).

Next: What must the voltage experiment compare fairly?

From graph mechanics to a physical voltage decision



Accepted request load, limits, scope

MPC / SAC candidate command

Numerical solve and fresh checks

Evidence for a next decision

Keep fixed: scenario, candidate budget, electrical constraints and objective.

Measure separately: numerical quality, failed checks, Llama's contribution and added time.

The next question is physical: what do active and reactive powers do to feeder voltage and current?

Matched budgets: PowerAgentBench-SS (2026 preprint). Numerical experiment: `Classroom_Labs.ipynb`, L2–L5.

Next: Which inverter action supports this feeder?

Voltage support: choose the next action, then test its consequences

Operator request

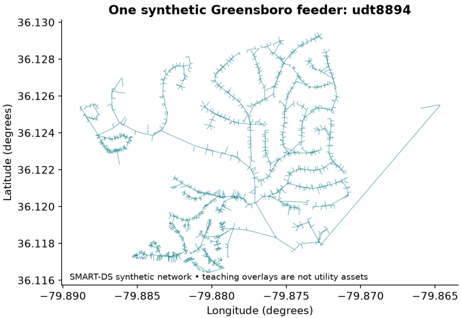
Keep the selected feeder within its modeled limits while using available PV.

Decisions

How much active power? How much reactive support? Which model should judge the proposal?

This chapter's experiment

MPC and SAC propose controls. Language models choose a bounded refinement. A fresh numerical check decides admissibility.


One synthetic Greensboro feeder: udt8894
36.130
36.128
36.126
36.124
36.122
36.120
36.118
36.116
Latitude (degrees)
−79.890
−79.885
−79.880
−79.875
−79.870
−79.865
Longitude (degrees)
SMART-DS synthetic network • teaching overlays are not utility assets

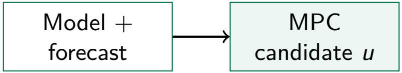
One SMART-DS synthetic Greensboro feeder: `uhs20_1247-udt8894`. The entire city is not modeled.

Palmintier et al., SMART-DS dataset (2020); `data/DATA_SOURCE.md`.

Next: Which methods can propose the same physical action?

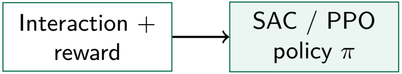
Model-based and model-free methods answer the same action question

Model-based: predict, then optimize



Optimization occurs at decision time. A wrong forecast or missing constraint can invalidate the proposal.

Model-free: learn, then infer



Training occurs before use. A new observation produces an action quickly; distribution shift can make it poor.

u: proposed control; π: learned observation-to-action rule. SAC is executed here; PPO is a discussed continuous-action alternative.

Fair comparison: same scenario, objective and acceptance limits. Both proposals return to a fresh power-flow check; an LLM response does not replace that check.

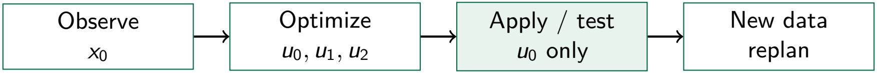
Rawlings, Mayne & Diehl, Model Predictive Control (2017). Haarnoja et al., SAC, ICML 2018; Schulman et al., PPO (2017).

Next: How does a planner use a forecast before acting?

MPC plans several moves and applies only the first

Read the symbols first: x_k is state, u_k is control and d_k is forecast demand at step k. H is the look-ahead horizon; ℓ scores one step; $\gamma = 0.98$ discounts later steps.

$$\min_{u_0,\ldots,u_{H-1}} \underbrace{\sum_{k=0}^{H-1} \gamma^k \ell(x_k, u_k)}_{\text{cost of the proposed sequence}} \quad \text{s.t.} \quad \underbrace{x_{k+1} = f(x_k, u_k, d_k)}_{\text{prediction model}}, \quad \underbrace{g(x_k, u_k) \le 0}_{\text{allowed operating region}} \quad .$$



Our implementation: $H = 3$ quasi-static power-flow snapshots linked by control-movement cost. The generic dynamic equation explains MPC; the tutorial uses algebraic snapshots and does not demonstrate transient stability or a closed-loop rollout.

Rawlings, Mayne & Diehl, Model Predictive Control (2017).

Next: What exactly should one control move minimize?

Make the objective interpretable before optimizing it

One two-number fleet command: $u = (a, b)$. For inverter i, S_i is its kVA rating, a is active participation and signed b is reactive participation; both are dimensionless. The one-step teaching cost is ℓ.

$$p_i = aS_i, \qquad q_i = bS_i, \qquad b < 0 \;\Rightarrow\; \text{reactive absorption.}$$

$$\ell = \underbrace{(1-a)^2}_{\text{unused active capability}} + \underbrace{0.05b^2}_{\text{reactive effort}} + \underbrace{10\ \text{mean}[(V-1.01)^2]}_{\text{voltage deviation}} + \underbrace{0.05\|u-u_{\text{prev}}\|_2^2}_{\text{command movement}}.$$

V: modeled voltage magnitudes (p.u.); 1.01: reference (p.u.); u_{prev}: previous command. Model scope determines which voltages enter the mean.

Predict: If we reduce $|b|$, which term certainly decreases? Does total cost necessarily decrease?

Only the reactive-effort term certainly decreases. Voltage, movement cost and feasibility can change too. These weights are teaching preferences under the declared availability assumption. A smaller one-step ℓ does not establish stability or field performance.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Next: Which constraints can overrule a low cost?

A low objective is useful only inside the admissible set

Symbols before limits: V_i is bus-phase voltage; I_e is branch current; lower/upper bounds define the contract. a, b are the fleet command; S_i is inverter rating.

Network admissibility

$$V_{\min} \leq |V_i| \leq V_{\max}, \quad |I_e| \leq I_e^{\max}.$$

Device admissibility

$$a^2 + b^2 \leq 1, \quad 0 \leq a \leq 1, \quad b_{\min} \leq b \leq b_{\max}.$$

The circle follows from $p_i^2 + q_i^2 \leq S_i^2$.

Evidence admissibility

Finite inputs and solver completion; dispatch tracking; matching case, control, model and evidence IDs.

The planner uses local sensitivities and a conservative capability polygon. The proposed action then returns to the selected numerical model.

Predict: Can a majority vote make an excessive current acceptable?

No. Voting selects a trial. The engineering gate determines whether it can be retained.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Rawlings, Mayne & Diehl, Model Predictive Control (2017).

Next: Which constraints can our chosen model see?

LinDistFlow3 and OpenDSS expose different parts of this feeder

	Python LinDistFlow3	**OpenDSS**
Relation	Linear three-phase approximation.	Nonlinear unbalanced power flow.
Network	MV; ideal aggregated LV demand.	Supplied MV/LV and transformers.
Meaning of PASS	Within the reduced contract.	Within the full-feeder snapshot contract.

Decode the voltage-drop model: $\mathbf{w}_i$ is squared phase voltage at parent bus i (V^2); j is its child; Z_{ij} is coupled impedance (Ω); $\mathbf{S}_{ij}$ is downstream phase power (VA).

$$\mathbf{w}_j = \underbrace{\mathbf{w}_i}_{\text{upstream voltage}} - \underbrace{2\,\mathrm{Re}\left[\overline{\beta} \odot \left(Z_{ij}(\beta \odot \overline{\mathbf{S}}_{ij})\right)\right]}_{\text{three-phase coupled voltage drop}}.$$

$\beta = (1, e^{-j2\pi/3}, e^{-j4\pi/3})$ gives nominal phase ratios; here j in the exponential is the imaginary unit. Bars mean conjugation; $\odot$ means entrywise multiplication; Re takes the real part.

Comparison boundary: different network coverage also changes which constraints can fail. This is not an accuracy-only comparison of equivalent models.

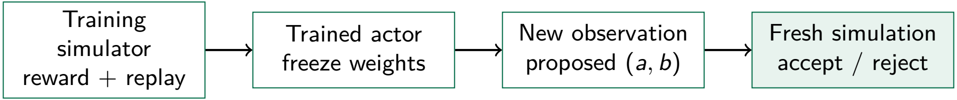
Gan & Low, three-phase distribution optimization (2014); EPRI OpenDSS documentation; `src/lindistflow3.py`.

Next: How does a learned actor produce an alternative?

SAC learns a continuous proposal; verification remains a separate step

Learning vocabulary: s_t is an observation, u_t an action, r a reward and π the actor. $\mathcal{H}$ measures policy entropy; α weights exploration and γ discounts later rewards.

$$\max_{\pi} \; \mathbb{E}\left[\sum_{t \geq 0} \gamma^{t} \left\{ \underbrace{r(s_t, u_t)}_{\text{task reward}} + \underbrace{\alpha\, \mathcal{H}(\pi(\cdot \mid s_t))}_{\text{exploration incentive}} \right\} \right].$$


Training simulator reward + replay

Trained actor freeze weights

New observation proposed (a, b)

Fresh simulation accept / reject

Executed artifact: 12,000 training transitions on a cubic surrogate fitted to 240 AC solves; 48 separate fit-validation samples. New scenarios change current-rating conditions: this is transfer, not retraining. A reward penalty does not prove constraint satisfaction.

Haarnoja et al., SAC, ICML 2018; Schulman et al., PPO (2017). Trained artifact: `Continuous_SAC_Training.ipynb`.

Next: What changes when we perturb the demand?

Lab 2: change demand and compare the same two numerical actions

Notebook L2 Fresh MPC solve, saved-actor inference and fresh numerical checks.

```
LOAD_SCALE = 1.0
RATING_FRACTION = 0.80
started = time.perf_counter()
voltage = lab.voltage_lab(LOAD_SCALE, RATING_FRACTION)
timings['L2 fresh MPC + SAC + checks'] = time.perf_counter() - started
```

Your turn: 7 minutes

Before running, predict whether a demand change increases PV curtailment or reduces it. Run the default experiment, then change one load multiplier. Compare objective, voltage/current margins and the gate result.

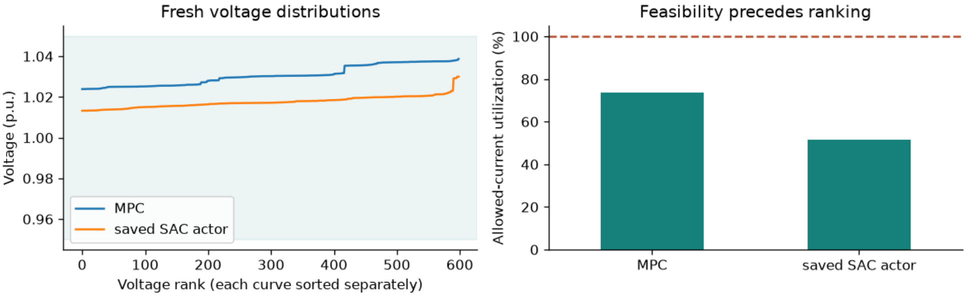
Record three things: the assumption you changed; the numerical response; whether the original recommendation still has matching evidence.

Do not infer: an action that was good at the original load is still good after the change.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Rawlings, Mayne & Diehl, Model Predictive Control (2017).

Next: Which margins explain the numerical result?

Read the margins before deciding which controller was better



Compare in this order: did the action pass the same gate? What objective did it achieve? What changed when the demand changed? What was excluded by the model?

Bridge: the numerical methods produced evidence. Now the LLM may choose a bounded next trial from that evidence.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Actual Llama responses can be useful and still contain unsupported claims

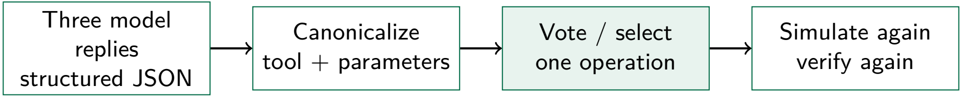
Model / time	Selected trial	Excerpt from its recorded public response
Llama 3.1 8B 1.867 s	Increase a by 0.0025	"Increasing active participation by 0.0025 may help reduce curtailment and improve the overall performance."
Llama 3.2 3B 0.905 s	Reduce $\|b\|$ by 10%	"Reducing reactive effort magnitude by ten percent may help improve the overall system performance and reduce curtailment"
Qwen 2.5 7B 1.844 s	Reduce $\|b\|$ by 10%	"may help improve voltage stability without significantly degrading active power performance"

Notebook L3 Original archived test 01; separate from the changed L2 classroom case.

Critique: less reactive effort alone does not establish less curtailment. These steady-state tests do not establish voltage stability.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Voting selects one physical operation, not the most persuasive paragraph



Three model replies structured JSON → Canonicalize tool + parameters → Vote / select one operation → Simulate again verify again

Two descriptions of the same (a, b) should not become two different actions. Canonical IDs make duplicates and ties reproducible.

Test 01: two models propose the same Q reduction; one proposes a PV increase. The majority selects the Q trial.

Why disagreement matters: it suggests another candidate to test. It does not quantify the probability that the grid action is safe.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Ensemble evaluation context: PowerAgentBench-SS (2026).

Next: Can feedback teach us which operation to test?

The LLM optimizer can learn which proposed operation to test

Select → observe → update. Here s is the evidence summary and a is a canonical proposed operation. $\phi(s, a)$ encodes their features; θ stores learned reward coefficients; A stores regularized feature information; β weights exploration. Score q chooses the next operation a^*.

$$q(a \mid s) = \underbrace{\theta^\top \phi(s, a)}_{\text{predicted reward}} + \underbrace{\beta \sqrt{\phi(s, a)^\top A^{-1} \phi(s, a)}}_{\text{training exploration bonus}}, \qquad a^* = \arg\max_a q(a \mid s).$$

After selection, observe reward r and update A and reward vector b:

$$A \leftarrow A + \phi\phi^\top, \qquad b \leftarrow b + r\phi, \qquad \theta = A^{-1} b.$$

This reward vector b is unrelated to inverter b. Frozen L4 evaluation uses predicted reward only, without updates.

Notebook L4 Load the frozen LinUCB selector, choose an operation and recheck it.

Boundary: the bandit selects among already generated responses. It neither selects which LLM to call nor trains LLM weights.

Li et al., LinUCB, WWW 2010; Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Next: What reward should the selector receive?

Reward learning and safety checking have different jobs

J_{inc} and J_{trial} are incumbent and trial current-step costs; r is feedback after a numerical trial.

$$r = \begin{cases} \min(1, \max(0, J_{\mathrm{inc}} - J_{\mathrm{trial}})/0.01) - 0.01, & \text{trial passes,} \\ -1.01, & \text{trial rejected.} \end{cases}$$

The penalty 0.01 is a fixed trial charge, not latency in seconds.

Learning design: 24 training contexts, 8 validation, 12 test; five training orders over the same data. Frozen validation/test: zero updates.

Your turn: 3 minutes

Run L4. Compare the selected trial with the majority vote. If the reward predicts improvement but fresh verification fails, which result should control the recommendation?

The gate. Retain a valid incumbent or escalate. Reward optimization does not create an operating permission.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

A rigorous result can show that the extra agent adds no benefit

MV test, 12 scenarios	Mean J	Full-feeder AC PASS	Accounted time
Numerical fixed refinement	0.03400656	0/12	3.327 s
Majority vote	0.03398479	0/12	Pool cost applies
Learned selector	0.03398479	0/12	8.119 s

Learned selection and voting tie on every held-out case. A posthoc fixed Q-reduction rule also reproduces these decisions without LLM calls.

What is demonstrated? Reproducible proposals, traceable refinement and explicit model-scope checking.

What is not demonstrated? Superior control from extra LLMs or learning. A simpler supervisor is a serious comparator.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Next: Will the same action pass a broader model?

Lab 5: ask a broader model to audit the recommendation

Notebook L5 Fresh MV check; optional fresh AC audit and separate AC replanning.

```
started = time.perf_counter()
scope_audit = lab.backend_audit(pool, run_opendss=RUN_OPENDSS)
timings[’L5 selected model scope checks’] = time.perf_counter() - started
display(scope_audit[’table’])
print(scope_audit[’optional_mode’])
```

Separate recorded pilot: 12 paired cases

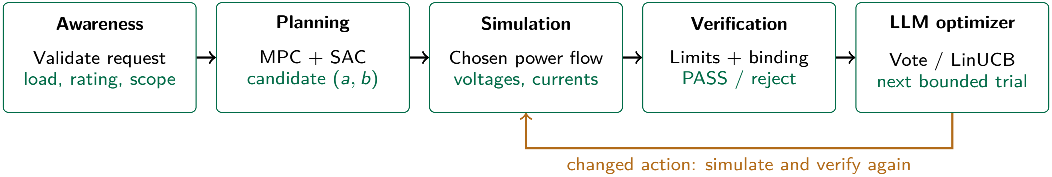
Final learned-workflow result	MV planning	OpenDSS planning
Selected-model final checks	12/12	12/12
Full-feeder AC checks	0/12	12/12
Human operating approval	Pending	Pending

Causal explanation: the network coverage and acceptance contract changed. The learned selector remained frozen. Do not attribute the broader-model benefit to Llama.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Next: Which stage produced each part of the evidence?

Voltage support: what each stage contributed


Awareness
Validate request
load, rating, scope
Planning
MPC + SAC
candidate (a, b)
Simulation
Chosen power flow
voltages, currents
Verification
Limits + binding
PASS / reject
LLM optimizer
Vote / LinUCB
next bounded trial
changed action: simulate and verify again

Evidence provenance: L2 performs fresh numerical solves and saved-actor inference. L3–L5 separately inspect the recorded response-selection pilot; its Llama replies are archived.

Operator handoff: retain the checked action, model scope and failed limits. In the recorded test, voting and learned selection tie; the reduced-model actions still fail the broader AC audit. The architecture exposes this boundary rather than proving better control.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Next: What changes when the decision is a topology, not a setpoint?

Ten-minute break: 82–92 minutes

Keep the notebook kernel running.

We have adjusted controls on a fixed network.

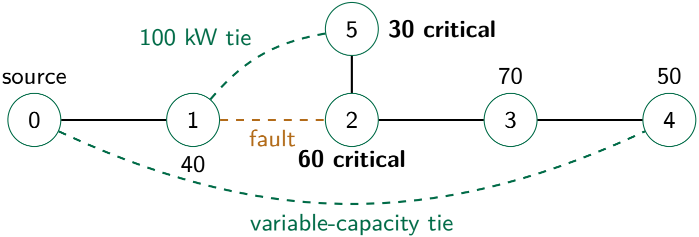
Next: a fault changes the available paths.

Which loads should the remaining network support?

Workshop timing: ten full minutes; no required work during the break.

Next: After the break, which loads should the remaining paths support?

After a fault, reconnecting service is a discrete decision


100 kW tie
5
30 critical
source
70
50
0
1
2
3
4
fault
40
60 critical
variable-capacity tie

Loads and tie capacities are in kW. This transparent six-node teaching graph is separate from the Greensboro feeder.

Predict: Can bus 4 carry power onward while its own load remains disconnected?

Yes. Energizing a bus and picking up its load are different decisions.

Chen et al., IEEE TPWRS (2018); Wang et al., radiality constraints (2020).

Next: How do we express pickup, energization and connectivity?

Restoration MILP: choose service while preserving a source path

Three binary choices: s_i picks up load i; y_i energizes its bus; z_e closes edge e. Demand d_i is in kW; priority $w_i = 10$ for critical loads and 1 otherwise. Source $y_0 = 1$.

$$\underbrace{\max \sum_i w_i d_i s_i,}_{\text{weighted service}} \qquad \underbrace{s_i \le y_i, \quad z_{ij} \le y_i, y_j}_{\text{pickup and edges need energized buses}} \quad , \quad z_{\text{fault}} = 0.$$

P_e is signed active flow; $\overline{P}_e$ is edge capacity (kW); A is the node-edge incidence matrix.

$$\underbrace{\sum_e A_{ie} P_e = d_i s_i \quad (i \neq 0),}_{\text{power reaches each served load}} \qquad \underbrace{|P_e| \le \overline{P}_e z_e}_{\text{open edge carries no flow}} \quad .$$

f_e is a fictitious connectivity commodity, not electricity; each energized non-source bus consumes one unit.

$$\sum_e A_{ie} f_e = y_i \ (i \neq 0), \qquad |f_e| \le 5 z_e, \qquad \underbrace{\sum_e z_e = \sum_i y_i - 1}_{\text{connectivity + edge count gives a tree}} \quad .$$

Scope: six-node active-power transport model; AC voltage, reactive power and switching chronology are omitted.

Chen et al., IEEE TPWRS (2018); Wang et al., radiality constraints (2020).

Next: What makes the comparison with a baseline fair?

A fair baseline must optimize the same feasible network

	Unweighted service baseline	Priority-weighted service
Objective	max $\sum_i d_i s_i$	max $\sum_i w_i d_i s_i$
Same constraints?	✓	✓
Same fault / capacities?	✓	✓
Exact solution method	Exhaustive enumeration	HiGHS MILP + enumeration check
Report	Total and critical kW	Total and critical kW

Interpretation: a difference caused by priority weights is an optimization-policy effect. It is not evidence that an LLM discovered a better optimizer.

Agent opportunity: turn a documented priority instruction into a validated parameter change; preserve the fault exclusions, action permissions and evidence chain.

Chen et al., IEEE TPWRS (2018); Wang et al., radiality constraints (2020). Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Lab 6: predict the effect of reducing tie capacity

Notebook L6 Fresh capacity sweep; independent enumeration checks the MILP.

```
CAPACITIES_KW = (40., 60., 80., 100., 120.)
started = time.perf_counter()
restoration = lab.restoration_sweep(CAPACITIES_KW)
timings['L6 fresh MILP sweep + independent enumeration'] = time.perf_counter() - started
display(restoration['table'].drop(columns='pickup'))
```

Your turn: 8 minutes

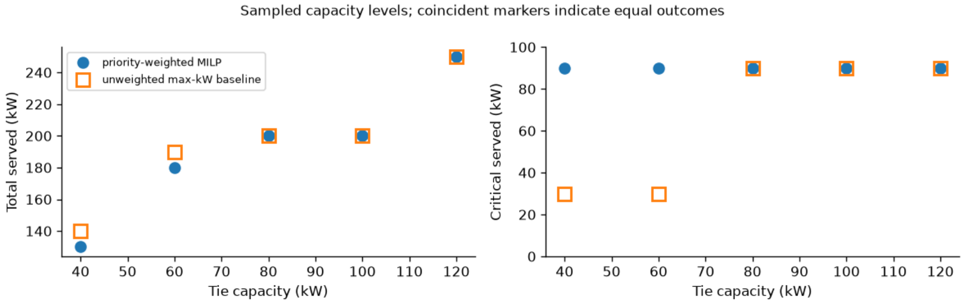
Predict total and critical service at 40, 60, 80 and 120 kW. Run the sweep; add a tighter rating. Compare weighted and unweighted solutions. Explain a flat region in the service curve.

Inspect: picked-up loads, closed edges, critical kW, total kW and enumeration agreement.

Why a sweep? One attractive topology cannot show how priorities and constraints interact.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Chen et al., IEEE TPWRS (2018); Wang et al., radiality constraints (2020).

Discrete service can stay constant while a rating changes



Explain the sampled service levels: loads are indivisible in this model. A tighter rating changes the solution only when it excludes a useful feasible pickup pattern.

At 100 and 80 kW in the priority formulation: 200 kW total, 90 kW critical. Bus 4 may remain energized while its own load is disconnected.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Next: Can this result still be used after the topology changes?

Lab 7: a changed topology needs a new evidence record

Notebook L7 Try to reuse a result after changing the case.

```
new_rest_request = {**rest_request, 'capacity_kw': 20., 'revision': 2}
print('Original contract:', lab.bound_to(rest_request, rest_candidate, rest_evidence))
print('Changed rating with old evidence:', lab.bound_to(new_rest_request, rest_candidate,
     rest_evidence))
print('Required response: HOLD old recommendation; request a fresh constrained solve.')
```

Your turn: 3 minutes

Change a capacity or pickup request while retaining the old evidence. Predict the verifier outcome. Recompute the evidence and repeat the handoff.

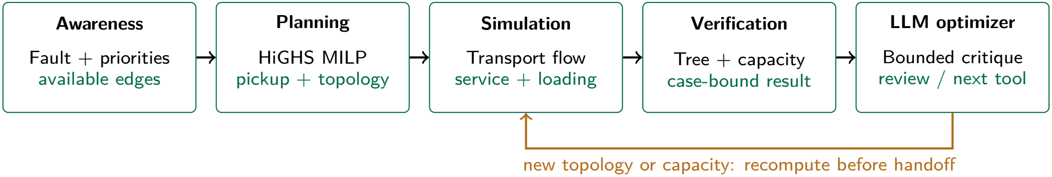
The safeguard: a passing result is bound to the case, control, model and contract that were actually checked.

The limit: this cannot establish field isolation, switch availability or protection coordination. Those need separate authorized evidence.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. State-prerequisite guards: PowerDAG (2026).

Next: Which stage must repair a stale handoff?

Use every workflow stage, but attribute the benefit correctly



Awareness
Fault + priorities
available edges

Planning
HiGHS MILP
pickup + topology

Simulation
Transport flow
service + loading

Verification
Tree + capacity
case-bound result

LLM optimizer
Bounded critique
review / next tool

new topology or capacity: recompute before handoff

Evidence provenance: L6–L7 freshly solve the six-node graph and test stale-evidence rejection. The five-stage Llama example is a separate recorded feeder catalog, inspected in L10.

Operator handoff: review isolation, permissions and model scope. In the recorded feeder comparison, the fixed and Llama routes obtain the same service objective; explanation adds traceability and latency, not demonstrated extra restored load.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Actual recorded five-stage reference: `Proposal_Complete_Workflow.ipynb`, P1–P8; `results/proposal_complete_workflow.json`.

Next: Can correctly bound evidence still describe the wrong state?

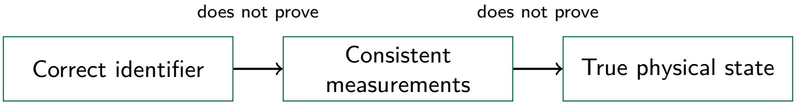
From restoration to cyber response: what if the evidence is wrong?

Established so far

We can optimize a permitted topology and reject evidence that belongs to the wrong case.

A harder question

What if the record matches the case, but the measurements inside it are corrupted?

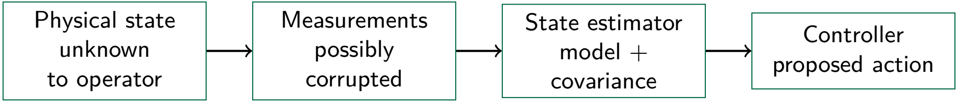

does not prove
does not prove
Correct identifier
Consistent measurements
True physical state

Next experiment: construct a measurement error that the usual residual test cannot see, then inspect its control consequence.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Next: How does an operator estimate a state from imperfect sensors?

Cyber response begins with an estimation problem



Physical state unknown to operator → Measurements possibly corrupted → State estimator model + covariance → Controller proposed action

Four-bus DC teaching system: three unknown non-reference voltage angles; eight measurements of flows, injections and one angle.

Two cases: an isolated biased sensor and a coordinated model-consistent perturbation.

The experimenter knows synthetic truth to score the outcome. The agent's operating gate does not receive that truth.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Next: What does a residual actually measure?

WLS measures consistency with a model and noise assumption

Symbols first: $x \in \mathbb{R}^3$ contains unknown angles (rad); $z \in \mathbb{R}^8$ contains sensor readings; H maps state to measurements. Noise ϵ has covariance R, which weights each sensor in its own units. WLS returns estimate $\hat{x}$.

$$z = Hx + \epsilon, \quad \epsilon \sim \mathcal{N}(0, R), \qquad \hat{x} = \arg\min_{x} \underbrace{(z - Hx)^\top R^{-1}(z - Hx)}_{\text{weighted mismatch between readings and model}} .$$

The residual r is the unexplained measurement mismatch; J is its dimensionless weighted energy.

$$r = z - H\hat{x}, \qquad J = r^\top R^{-1} r, \qquad \underbrace{J > \chi^2_{5,0.99} = 15.0863}_{\text{five residual degrees of freedom: } 8-3} \quad \Rightarrow \ \text{alarm}.$$

What this means: readings inconsistent with the assumed model and noise. It does not identify malicious intent.

The 1% nominal false-alarm level assumes a correct linear model, Gaussian noise and full observability. It is not a measured field false-alarm rate.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Next: Can the estimate change while the residual stays the same?

A coordinated perturbation can change the estimate without the residual

Define c as a state displacement, a as a measurement perturbation and primes as changed quantities. Choose a coordinated pattern $a = Hc$:

$$z' = z + Hc.$$

For the same full-rank linear WLS model, $\hat{x}' = \hat{x} + c$. Therefore

$$r' = z' - H\hat{x}' = z + Hc - H(\hat{x} + c) = z - H\hat{x} = r.$$

Same residual, different estimated state.

A controller can confidently choose an action for the wrong operating point. More eloquent interpretation of the same residual does not recover the missing information.

This construction assumes the attacker can alter the required measurement pattern and knows the relevant linear model. Real restrictions can limit that capability.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Lab 8: sweep the attack size and watch two different signals

Notebook L8 Fresh WLS experiment for isolated and model-consistent perturbations.

```
ATTACK_SCALES = (0., .25, .5, 1., 1.5)
started = time.perf_counter()
cyber = lab.cyber_sweep(ATTACK_SCALES)
timings['L8 fresh WLS attack sweep'] = time.perf_counter() - started
display(cyber['table'])
```

Your turn: 7 minutes

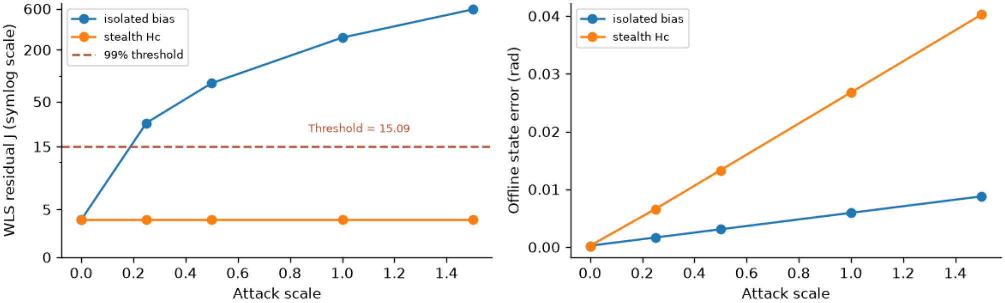
Predict the residual and state-error curves for both perturbations. Run the sweep. Find an example where the residual stays below threshold while state error grows.

Keep two plots separate: a detector sees residual statistics; the experimenter evaluates estimation error using synthetic truth.

Question: what can the agent truthfully conclude from “no residual alarm”?

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

A missed alarm can coexist with a large estimation error



Read left to right: residual consistency and state accuracy answer different questions. A model-consistent perturbation preserves the former while degrading the latter.

Next requirement: an independent, justified information source or a safe escalation path.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Protected measurements enable a constrained recovery action

Information comes first: $\hat{x}_{\mathcal{T}}$ is the state estimated from three explicitly protected angle readings. The fixture assumes that trust; neither WLS nor the LLM establishes it.

Action and physics: u is added node-2 injection, balanced at the source (p.u.); F maps angles to branch flows; b_u is branch-flow sensitivity to u.

$$\min_{0\leq u\leq 0.5} \quad \underbrace{\tfrac{1}{2}u^2}_{\text{smallest corrective injection}} \quad \text{s.t.} \quad \underbrace{|F\hat{x}_{\mathcal{T}} + b_u u|}_{\text{predicted branch-flow magnitudes}} \leq \underbrace{\overline{f} - 0.01}_{\text{limits minus planning margin}} .$$

$\overline{f} = (0.35, 0.18, 0.35, 0.35)$ p.u. gives each branch's limit. The 0.01 p.u. margin is an engineering choice, not a calibrated chance constraint.

Predict: Do three independent readings for three unknown states leave a residual consistency test?

No redundancy remains. Rank 3 enables estimation, but residual degrees of freedom are zero. If the protected evidence is unavailable, preserve uncertainty and escalate.

Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009. Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Lab 9: verify the control consequence under the same synthetic truth

Notebook L9 Compare corrupted-state and protected-state control.

```
without_trust = lab.trusted_response(cyber, protected_available=False)
with_trust = lab.trusted_response(cyber, protected_available=True)
```

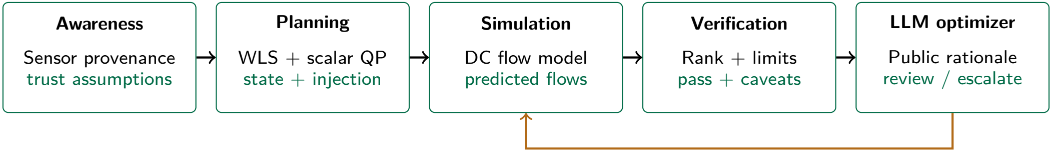
Original toy fixture	u (p.u.)	Actual flow 1–2 (p.u.)
Stealth-corrupted estimate	0	0.24000
Protected estimate	0.12169	0.17046

Your turn: 5 minutes

Run L9. Check the 0.18 p.u. limit and all other branches. Explain why protected information, rather than an LLM label, changes this result.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

The cyber workflow must preserve uncertainty at each handoff



Awareness
Sensor provenance
trust assumptions

Planning
WLS + scalar QP
state + injection

Simulation
DC flow model
predicted flows

Verification
Rank + limits
pass + caveats

LLM optimizer
Public rationale
review / escalate

changed estimate or action: test the new consequences

Evidence provenance: L8–L9 freshly evaluate the DC fixture; synthetic truth is for scoring, not given to the operating gate. L10 reads the separate archived five-stage Llama records.

Operator handoff: retain the unverified trust assumptions. The recorded fixed and Llama routes use the same numerical control; improved attack detection is not established by the language layer.

Actual recorded five-stage reference: `Proposal_Complete_Workflow.ipynb`, P1–P8; `results/proposal_complete_workflow.json`. Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Next: Which failure boundary is common across the three cases?

The three cases teach three different failure boundaries

Case	What a naive workflow can miss	Evidence that addresses it
Voltage	A reduced model omits a binding limit.	Broader-model validation and explicit scope.
Restoration	A good plan is applied after topology changes.	Fresh case-bound simulation and permissions.
Cyber	Plausible measurements describe the wrong state.	Justified trusted information or escalation.

Common architecture: choose tools, retain provenance, enforce contracts, expose limitations and involve the operator.

Research question: when does learning improve this architecture enough to justify its additional time, cost and failure modes?

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Recent research asks when coordination is worth its cost

Research result: Kim et al. study 260 configurations across six agent benchmarks. Collaboration depends on task structure and the capability of the single-agent baseline.

Design choice	Possible benefit	What our experiment must measure
Parallel specialists	Explore different candidates	Distinct valid actions per budget
Central coordinator	Preserve shared state and check handoffs	Stale evidence and rejected calls
Learned response selector	Allocate a trial using past feedback	Held-out reward and selection cost

Return to L4 and E5. The learned selector ties majority voting. A post-hoc fixed reactive-power rule produces the same actions. This result makes the added model calls a research question.

Discussion, 30 seconds

If three models propose the same action, what additional information has the ensemble supplied?

Kim et al., agent-system scaling, v3 (2026). Published title: Capable language models can outgrow the benefits of collaboration, Nature Machine Intelligence (2026). Tutorial evidence: E5, saved voltage pilot.

Next: How can an effective procedure carry over to a new task?

Reusable skills make the procedure explicit

A recent power-system example. Etingov and Biswas combine reusable skills, subagents and MCP-connected PSSE tools for power-flow and dynamic-study tasks.

A reusable procedure needs	Voltage-support example for this classroom
Entry conditions	Current topology, trusted measurements and declared limits
Method and tools	MPC or SAC proposal, model execution and fresh verification
Evidence to return	Candidate ID, model scope, constraint margins and objective
Exit or escalation	Verified recommendation, or a reason to hold for review

Code connection, L1. `bind_evidence` ties the result to its request and candidate. `bound_to` rejects a changed load or command.

A small experiment

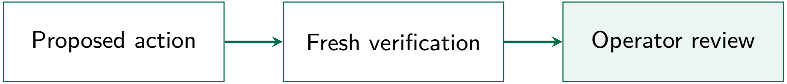
Remove the model scope from a handoff. Decide which later claim can no longer be justified.

Etingov and Biswas, Skill-Based AI Agents for Power-System Studies, v2 (1 October 2026), preprint. Context design: Anthropic, Effective context engineering (2025), engineering guidance.
Table: proposed classroom procedure.

Next: Which decisions require an enforced authority boundary?

Control-room authority belongs in executable rules

2026 governance study: an external layer enforces tool permissions, step budgets, operator approval and numerical provenance. Its prototype reports 93.7% task success and 100% rule compliance in 590 runs.



Proposed action → Fresh verification → Operator review

A changed candidate, state or operating limit invalidates the prior review.

The rule can hold	**The task can still fail**
An approval gate blocks unapproved actions.	The proposal may name the wrong equipment.
A number points to a backend result.	The agent may choose the wrong source.

Lab L12: `operator_packet` checks the required scope. `acknowledge` checks readback identity. The endpoint remains advisory.

Mylonas, Foti and Varvarigos, Governance-Aware Agentic Digital Twin for TSO Control Rooms (18 September 2026), preprint, §6. Diagram: tutorial review boundary.

Next: How could we test the benefit of this orchestration fairly?

The next study should isolate workflow value

A testable question: does agentic orchestration complete unfamiliar operator requests more reliably with the same engineering tools and budget?

Comparison	Hold fixed	Measure separately
Scripted workflow vs. Llama orchestration	Models, observations, limits and available tools	Correct intent, completed stages and unsupported claims
One LLM vs. several roles	Base model and total tool/token budget	Handoff errors, recovery and latency
Fixed vs. vote vs. learned selection	Response pool, trial budget and frozen test set	Feasibility, objective and selection overhead

Stress extension: change forecast uncertainty or the model scope, then repeat the audit. Keep test outcomes out of prompts and learned-selector updates.

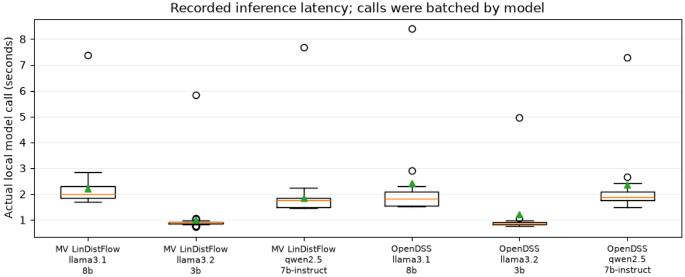
Exit question

Which measured outcome would justify adding one more agent to your control-room workflow?

Evidence-based scoring: PowerAgentBench-SS (June 2026). Uncertainty-aware orchestration: CONDUCTOR (June 2026). Table: proposed evaluation design, not a reported experiment.

Next: What time did the extra coordination actually cost?

Latency must be measured where the work actually happens



Recorded experiment: 168 real local-model calls. MV mean accounted serial time: learned selection 8.119 s; fixed numerical refinement 3.327 s.

Separate: solver time, LLM load/inference, retries and final verification. Cached notebook display time is not inference latency. The AC transfer reused MPC planning and reports that component separately.

Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Next: What can we inspect in one real local Llama call?

Lab 11: one optional local Llama call, with visible provenance

Notebook L11 Default is offline; explicitly enable the optional local call.

```
client = ProposalLlama(journal, model='llama3.1:8b')
live_reply = client.ask('Optimization Agent', pool['context']['public_context'],
                        list(pool['context']['catalog']))
```

Actual validation, 17 October 2026: Llama 3.1 8B chose `RELAX_Q_10_PERCENT` on the original test_01 context.

Excerpt from the returned public reason

"the RELAX_Q_10_PERCENT operation has the potential to reduce reactive effort magnitude by ten percent, which may improve the cost."

Observed time: 34.113 s client elapsed; Ollama reported 16.967 s loading, 15.192 s prompt processing and 1.916 s generation.

This is one call, shown from its saved record. The default L11 makes no new call. Inspect the complete response and mode in the notebook; a reason still requests a fresh trial.

Ollama chat API; Llama 3 model report (2024); actual record: `results/classroom_optional_llama_validation.json`; Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Evaluate the failure, the recovery and the final outcome separately

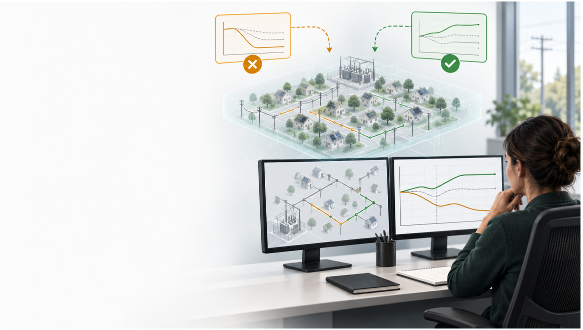
Metric	Question	Common mistaken conclusion
Task completion	Did the intended study finish?	Valid JSON means success.
Electrical feasibility	Which constraints passed, on which model?	Solver convergence means safe.
Recovery	Was the failed attempt repaired within budget?	More retries mean intelligence.
Claim support	Does each numerical statement match evidence?	A fluent reason is evidence.
Operator benefit	Did correct decisions improve with acceptable workload?	A dashboard proves human benefit.

Our evidence supports an executable teaching workflow. A field claim would need broader scenarios, stronger models and a study with operators.

PFAgent (2026); PowerAgentBench-SS (2026); MAST failure taxonomy.

Next: What evidence should reach the operator?

The control room receives a recommendation with an evidence trail



Concept illustration; AI-generated, not a deployed control room.

What the operator needs

Requested action and exact equipment or candidate ID.

Current case, model scope and constraint margins.

Failed alternatives, unresolved assumptions and age of evidence.

Concise public rationale, with each numerical claim tied to a tool result.

Deployment concept: advisory service beside existing operational applications; authorized procedures govern any later field action.

Governance-aware TSO digital twin (2026); illustration provenance in `figures/Proposal_Illustration_Provenance.md`.

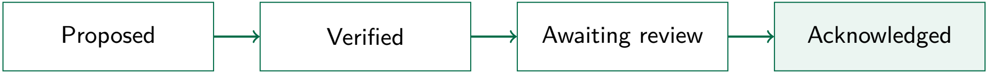
Lab 12: acknowledgment is a state transition, not a generated sentence

Notebook L12 Build an operator packet and test the readback gate.

```
wrong_id = lab.acknowledge(mv_scope, ’wrong-id’, ’Student operator’)
readback = lab.acknowledge(mv_scope, mv_scope[’recommendation_id’], ’Student operator’)
```

Your turn: 4 minutes

Try a wrong candidate ID, then the correct readback. Change the requested control after review. Decide whether the old acknowledgment should remain valid.



Proposed → Verified → Awaiting review → Acknowledged

This is a local advisory exercise. No state in this notebook dispatches equipment.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Runtime authority: Mylonas et al. (2026).

Next: Can we reconstruct every stage from one saved trace?

One trace should answer what happened at every stage

Notebook L10 Compare real saved stage records and public responses.

```
archived_reasons = []
for case, backend in [('voltage', 'lindistflow3'), ('restoration', 'lindistflow3'), ('cyber_stealth'
    , 'dc')]:
    run = lab.archived_stage_replies(case, backend)
```

Inspect	What the answer should expose
Input	Case, measurement assumptions, allowed operations and model scope.
Method	Which optimizer, policy, simulator or language call actually ran.
Output	Candidate ID, returned values, constraint failures and public reason.
Cost	Measured tool/call time, including failed attempts.
Disposition	Retained, rejected, refined or escalated; the next required step.

Trace labels matter: classroom fresh solve, archived actual LLM response and optional live call are different evidence modes.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Actual recorded five-stage reference: `Proposal_Complete_Workflow.ipynb`, P1–P8; `results/proposal_complete_workflow.json`.

Next: Which missing evidence changes the operator decision?

Capstone: would you approve, hold or request another study?

Card	Evidence presented	Your decision
A	MV PASS; full-feeder AC FAIL; Llama says “ready”.	Review / hold / study?
B	Restoration MILP PASS; equipment isolation has no supporting record.	Review / hold / study?
C	No residual alarm; protected measurement set is unavailable.	Review / hold / study?

Task: identify the missing fact and the appropriate tool or human step. Do not answer with a model’s confidence score.

Debrief: A requires broader-model repair; B requires authorized isolation evidence; C requires justified independent information or escalation. None supports automatic dispatch.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Schweppe & Wildes, state estimation (1970); Liu, Ning & Reiter, false-data injection, CCS 2009.

Next: Which architecture fits each kind of operator task?

Match the architecture to the operator task

Task condition	Useful architecture	Evidence required
Stable repeated calculation	Fixed workflow + numerical checks	Correct model and tested contract.
Varied language requests	LLM tool selection + typed state	Intent accuracy and supported claims.
Several independent studies	Bounded specialist roles	Handoff quality and real parallel benefit.
Competing refinements	Vote or learned selector	Held-out gain at equal trial budget.
High-consequence action	Human review + enforced authorization	Correct action, scope, provenance and permissions.

Design principle: add complexity only when it solves a measured failure of the simpler workflow.

Kim et al., scaling agent systems (2026); governance-aware TSO study (2026).

Next: Can you explain one complete case using the five stages?

Explain the full workflow in sixty seconds

Awareness: establish the current case and what is known.

Planning: choose an appropriate numerical method and form candidates.

Simulation: calculate each candidate's consequences in a declared model.

Verification: check physical limits, identity, evidence and permissions.

LLM optimizer: propose a bounded next trial, then return through simulation and verification.

Operator review: inspect the recommendation and unresolved assumptions before any authorized action.

Partner exercise: use one of today's cases and name the actual method and output at every stage.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit.

Three checks for scientific claims about agentic control

1. Is the comparison matched?

Same observations, model coverage, limits, tools, candidate budget and test scenarios?

2. Which component caused the difference?

Better optimizer, broader model, protected information, language interpretation or coordination?

3. What was actually measured?

Constraint compliance, objective, task completion, claim support, latency or human performance?

Today's result: the framework makes these questions executable. Extra language models and learned selection did not establish general superiority in the recorded pilot.

PowerAgentBench-SS (2026); Actual recorded voltage pilot: `LLM_Response_Selection.ipynb`, E1–E8; raw calls and numerical records in `results/ensemble_voltage_pilot/`.

Next: What single assumption would you change in the next experiment?

Your next experiment: change one assumption and keep an audit trail

Extension	Change one thing	Keep fixed
Voltage	Forecast uncertainty or a current margin	Controls, model scope and trial budget.
Restoration	Priority weights or one tie rating	Fault, graph and baseline constraints.
Cyber	Sensor trust or noise assumptions	Physical truth and scoring protocol.
Agent design	One role, prompt or selector feature	Tools, test split and numerical gates.

Deliverable: one plot, a baseline comparison, a trace explaining the result and a statement of what remains unproven.

After class: the appendix contains derivations, full code excerpts, recorded responses and training details. It is reference material outside the 180-minute route.

Executed classroom experiment: `Classroom_Labs.ipynb`; source: `src/classroom_labs.py`. Teaching assumptions are explicit. Reading guide: `appendix/Agentic_AI_Research_2026.md`.

Next: Choose an extension, then use the appendix when you need its derivation.

Technical appendix: use after the three-hour class

The taught sequence ends at frame 72. The following pages preserve deeper derivations, implementation excerpts, recorded studies and full mathematical definitions.

G0–G5: real LangGraph teaching example, with declared toy current.

L0–L12: current classroom notebook, used in the lecture.

S/W/M/V/A/R/C/F: original numerical workshop.

P: archived five-stage workflow; **E:** archived response-selector pilot; **T:** archived latency analysis.

These are separate experiments. Their numerical results, model scopes and call timings must not be mixed into a single experiment.

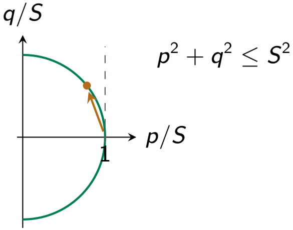
Exact mapping and source hashes: `Slide_Notebook_Map.md` and `checkpoints.json`.

Next: Which physical limit couples active and reactive power?

Two controls share one inverter capability

Question

At active participation $p/S = 1$, can the inverter also supply reactive power?



Active power p (kW): delivered PV production; nonnegative here.

Reactive power q (kvar): injection if positive, absorption if negative. Both affect voltage and current.

S (kVA) is the inverter rating; p/S and q/S are dimensionless. The circle is a device limit, not a network-feasibility test.

Reason

No. More reactive support can require less active output. We must check the resulting network currents as well as voltage.

Implemented device domain: `src/standard_formulations.py`; physical model: EPRI OpenDSS. Diagram illustrates capability, not a measured point.

Next: Which limits turn the request into an optimization problem?

Turn the operating request into a checkable problem

General Volt/VAR OPF: choose active/reactive setpoints p, q to minimize

$$C_{\mathrm{PV}}(p) + C_v(v) + C_q(q), \qquad v_{\min} \le v_{i\phi} \le v_{\max}, \quad |I_{\ell\phi}| \le \rho I_{\ell\phi}^{\mathrm{rated}}, \quad p_g^2 + q_g^2 \le S_g^2.$$

The selected three-phase power-flow equations also hold; $0 \le p_g \le p_g^{\mathrm{av}}$ and device reactive bounds apply.

Symbol	Definition and physical interpretation
i, ϕ, ℓ, g	Bus, available phase, line/conductor, and inverter indices.
$v_{i\phi},\ v_{\min},\ v_{\max}$	Voltage magnitude and allowed bounds (p.u.); here 0.95 and 1.05.
$I_{\ell\phi},\ I_{\ell\phi}^{\mathrm{rated}}$	Predicted current magnitude and nominal rating (A). $\rho \in (0, 1]$ is the permitted rating fraction.
$p_g, q_g, S_g, p_g^{\mathrm{av}}$	Active output (kW), reactive output (kvar), rating (kVA), and available PV (kW).
$C_{\mathrm{PV}}, C_v, C_q$	Weighted penalties for curtailment, voltage deviation and reactive effort; weights reconcile units and state the preference.

Agent boundary: preserve the objective, limits and permissions; numerical tools compute and check the setpoints.

General three-phase formulation: Gan and Low (2014), `https://arxiv.org/abs/1406.3054`. Full objective, units and constraints are in the mathematical appendix.

Next: Which quantities can this particular experiment control?

What does this teaching experiment actually control?

All 930 inverters share **two dimensionless controls**; this is a reduced continuous OPF:

$$p_g = aS_g, \quad q_g = bS_g, \qquad 0 \le a \le 1, \quad |b| \le 0.44, \quad a^2 + b^2 \le 1.$$

Here g identifies an inverter, S_g is its kVA rating, p_g is kW, and q_g is kvar. The high-irradiance overlay assumes available active power equals rated kVA.

$$C(a, b) = \underbrace{(1 - a)^2}_{\text{PV curtailment}} + \underbrace{0.05b^2}_{\text{reactive effort}} + \underbrace{\frac{10}{N_v} \sum_{j=1}^{N_v} (v_j - 1.01)^2}_{\text{voltage regulation}}.$$

C is a dimensionless teaching cost, not money; its coefficients are declared preferences. v_j is the jth modeled energized node-phase voltage in p.u.; N_v counts those samples; 1.01 p.u. is the target.

How it is solved: finite differences form local sensitivities; bounded QP steps use an inscribed 32-sided capability polygon. A fresh backend solve checks each candidate. A feasible local improvement is not a global AC optimum.

Teaching implementation: `src/standard_formulations.py`; general phase-coupled formulation: Gan and Low (2014).

Next: What operating point does the numerical optimizer return?

V1: ask the optimizer for a better operating point

Notebook V1 Run the complete cell; this line performs the optimization.

```
voltage = run_voltvar(BACKEND)
```

Selected MV model	Total p (MW)	Total q (MVAr)	$v_{\max}$ (p.u.)	Current (%)
Before	7.850	0.000	1.0480	60.8
After	7.727	-1.249	1.0370	65.3

Current = maximum line/conductor current as a percentage of nominal rating; $v_{\max}$ is the largest modeled voltage magnitude.

What to look for

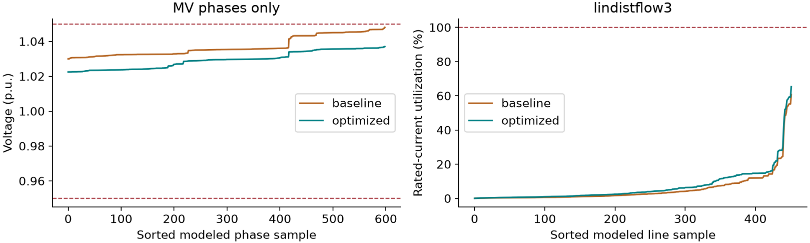
The objective falls from 0.00760 to 0.00549. Both points pass the MV check; the optimized point trades some active power and reactive effort for voltage regulation.

Recorded LinDistFlow result: `results/standard_formulations.json`; code checkpoint V1. Model: Gan and Low (2014).

Next: Which constraint can become worse when voltage improves?

V2: better voltage can come with higher current

Notebook V2 Run the plot and compare the two curves in each panel.



What to look for

Voltage moves toward the target; current increases. Both remain within the **modeled** limits.

Actual snapshot plot: `results/workshop_run_lindistflow3.json`, checkpoint V2. Model: Gan and Low (2014).

Next: What changes if only reactive power can move?

Hold active power fixed; change only reactive power

Your turn: 3 minutes

Predict first: how will absorption affect voltage and current at the same active output?

1. Create a scratch `PQBackend(BACKEND)` for the selected model.
2. Solve $u_1 = (0.8, 0)$, then $u_2 = (0.8, -0.1)$ using that scratch model.
3. Compare achieved P/Q, maximum voltage and current; keep V1's result unchanged.

The dimensionless vector $u = (a, b)$ sets active and reactive participation; u_1, u_2 label the two trials. Explain the observed tradeoff in one sentence. A point with lower voltage can still have higher current.

Exercise uses `src/standard_formulations.py`; models: EPRI OpenDSS / Gan and Low (2014).

Next: Will the same control pass the fuller model?

V3: does the same control pass a fuller audit?

Notebook V3 Read the recorded comparison; do not change the control between checks.

Question

Does an MV-only pass establish an acceptable full-feeder AC result?

The same aggregate PV outputs are $p = 7.727$ MW and $q = -1.249$ MVAr; negative q means absorption.

Evaluation of that point	Max. voltage (p.u.)	Nominal current (%)	Result
MV-only LinDistFlow	1.037	65.3	Pass
Full-feeder OpenDSS audit	1.087	288.2	Fail

Reason

No. The MV model omits LV constraints. The full OpenDSS optimizer finds a different checked point: 2.904 MW and 0.346 MVAr.

Identical-control audit and separate optimized AC point: `results/standard_formulations.json`; EPRI OpenDSS.

Next: How does a horizon plan extend a single operating point?

MPC chooses a continuous P/Q sequence

Model predictive control (MPC) plans $H = 3$ quasistatic steps jointly, then proposes the first move:

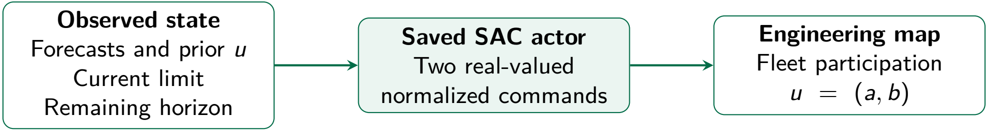
$$\min_{u_0,u_1,u_2} \sum_{k=0}^{2} \gamma^k \ell_k, \qquad \ell_k = (1 - a_k)^2 + 0.05 b_k^2 + \frac{10}{N_v} \sum_{j=1}^{N_v} (v_{j,k} - 1.01)^2 + 0.05 \| u_k - u_{k-1} \|_2^2.$$

Quantity	Meaning / declared value
$k, H, u_k = (a_k, b_k)$	Step index, horizon length, and continuous dimensionless inverter settings. u_{-1} is the given previous command.
$\gamma = 0.98$, ℓ_k	Dimensionless discount and stage cost. Movement is the squared Euclidean change in (a, b) between commands.
$v_{j,k}$, N_v	Predicted voltage of modeled node-phase j at step k (p.u.); number of voltage samples. Target: 1.01 p.u.
Forecast and limits	Known load multipliers; every planned step obeys voltage, allowed current and device constraints. No storage dynamics are included.
Numerical method	Local sensitivity QPs, bounded steps and fresh horizon checks; retain the last feasible improvement if a step fails.

Interpretation: one coupled horizon problem, not three unrelated OPFs. This experiment inspects one planning decision; it neither demonstrates closed-loop stability nor certifies a local/global optimum.

MPC: Rawlings, Mayne and Diehl (2017); distribution voltage MPC: Maharjan et al. (2021). Actual implementation: `src/continuous_control.py`.

SAC maps the operating state to continuous controls



Observed state
Forecasts and prior u
Current limit
Remaining horizon

Saved SAC actor
Two real-valued normalized commands

Engineering map
Fleet participation
$u = (a, b)$

$$a = \frac{z_1 + 1}{2}, \qquad b = z_2 \min\left(0.44, \sqrt{1 - a^2}\right), \qquad z_1, z_2 \in [-1, 1].$$

z_1, z_2 are the actor's dimensionless normalized outputs; $a = p/S$ and $b = q/S$ are fleet participation ratios. The square root limits reactive headroom as active output rises; 0.44 is the declared reactive-ratio bound.

$u = (a, b)$ is the previous/current command; current allowance is a fraction of nominal rating. All seven policy inputs are dimensionless: three remaining load multipliers, previous a, b, allowance, and remaining horizon divided by three.

Boundary: the map respects inverter capability; fresh electrical checks determine network feasibility.

SAC: Haarnoja et al. (2018). Exact action map and observation: `src/continuous_control.py`.

Next: Which feedback trains the policy?

What does SAC learn to reward?

The training reward uses the same engineering stage cost plus penalties on a fitted response surface:

$$r_t = -\ell_t - 40(e_v/0.05)^2 - 10e_I^2 - 2\,\mathbf{1}\{e_v > 0 \text{ or } e_I > 0\}, \quad e_v = [0.95 - \widehat{v}_{\min}]_+ + [\widehat{v}_{\max} - 1.05]_+, \quad e_I = [\widehat{\iota} - 1]_+.$$

At training step t, r_t is dimensionless reward and ℓ_t penalizes curtailment, reactive effort, voltage deviation and movement. e_v is voltage-band excess (p.u.); e_I is current-ratio excess. Hats denote surrogate predictions; $\widehat{\iota}$ is maximum current divided by nominal rating. $[x]_+ = \max(x, 0)$; indicator $\mathbf{1}$ equals one for a violation, otherwise zero.

$$\max_{\pi} \ \mathbb{E}_\pi\left[\sum_{t=0}^{2} \gamma^t\{r_t + \alpha\mathcal{H}(\pi(\cdot \mid o_t))\}\right], \qquad \gamma = 0.98, \quad \alpha = 0.005.$$

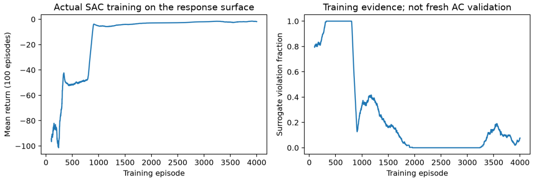
π is the stochastic normalized-action policy; o_t is its seven-value observation; $\mathbb{E}_\pi$ averages trajectories. $\mathcal{H}$ is differential entropy in normalized-action coordinates; α weights exploration and γ discounts future reward. Both constants are fixed for this run.

Learning mechanism: an actor proposes actions; twin critics estimate return from replayed transitions. Training samples actions; evaluation uses tanh of the Gaussian mean. Penalty learning does not prove constraint satisfaction.

SAC: Haarnoja et al. (2018); exact reward and configuration: `src/continuous_control.py`, `models/continuous_sac/training.json`.

Next: Does a favorable reward establish engineering feasibility?

Training reward and fresh engineering checks answer different questions



In plain language

The reward penalizes operating cost and predicted violations. Fresh OpenDSS or LinDistFlow solves evaluate each proposed action independently.

Actual run: 12,000 reward interactions, 11,400 gradient updates, seed 23. The cubic response surface uses 240 AC fit points and 48 separate fit-validation points; current-fraction MAE is 0.041.

Training uses nominal current allowance and loads in $[0.7, 1.2]$ times base demand. The 80% allowance test changes that constraint; no MPC action labels are used.

SAC learning: Haarnoja et al. (2018). Executed training provenance: `models/continuous_sac/training.json`; reward: `src/continuous_control.py`.

Next: Why choose SAC, and when could PPO be appropriate?

Why SAC here, and where would PPO fit?

	SAC	PPO
Experience	Reuses a replay buffer	Trains from collected policy rollouts
Learning principle	Reward plus entropy regularization	A clipped policy-update objective
Continuous P/Q	Continuous actor is implemented	A continuous policy is possible
Evidence here	Saved trained actor and fresh tests	Discussed alternative; not executed

Takeaway

The workflow contract can accept another policy. A method name does not bypass the common verifier.

SAC: Haarnoja et al. (2018). PPO: Schulman et al. (2017). Executed policy provenance: `models/continuous_sac/training.json`.

Next: What do MPC and the saved SAC actor propose for one case?

V4: produce an MPC plan and a SAC proposal

Notebook V4 Run the complete cell for the three authored operating cases.

```
policy = load_continuous_policy()
plans = {key: optimize_continuous_mpc(case, BACKEND) for key, case in continuous_casebook.items()}
proposals = {key: sac_proposal(case, policy) for key, case in continuous_casebook.items()}
```

MPC output
Three continuous controls
and fresh horizon evidence

SAC output
One continuous proposal
from the saved actor

What to look for

Find a, b, the model scope, and the method that produced each value before comparing outcomes.

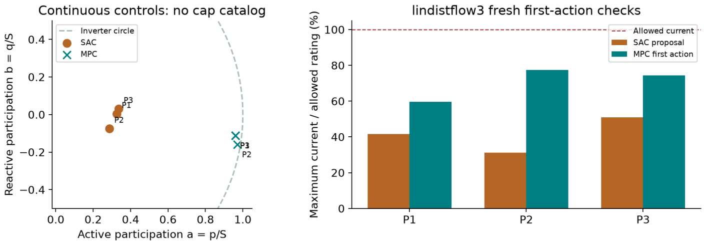
Actual numerical study: `results/continuous_control.json`; code: `src/continuous_control.py`. SAC: Haarnoja et al. (2018).

Next: Does each proposed action pass a fresh check?

V5: a continuous proposal still needs a fresh check

Notebook V5 Solve the exact SAC point in the selected model and inspect its failed checks.

```
raw = plant.solve(proposals[key]['candidate']['control'], case.forecast_loads[0], case.
 rating_fraction)
```



Read each backend separately. A pass applies to the first action and that model's scope.

Fresh evaluation: `results/continuous_control.json`; common checks: `src/continuous_control.py`. Model references: EPRI / Gan and Low (2014).

Next: How do the decision variables change after a fault?

Which constraints make a restoration plan feasible?

For bus i and available branch $e = (i, j)$, let $y_i, s_i, z_e \in \{0, 1\}$ indicate energized bus, picked-up load and closed branch. Let P_e be signed active flow (kW) and $\overline{P}_e$ its capacity (kW).

$$s_i \le y_i, \quad z_e \le y_i, y_j, \quad y_0 = 1, \qquad -\overline{P}_e z_e \le P_e \le \overline{P}_e z_e, \qquad \sum_e A_{ie} P_e = P_i^d s_i \quad (i \ne 0).$$

A is the node–branch incidence matrix: $+1$ for arrival, -1 for departure. P_i^d is bus demand (kW); source bus 0 supplies the total. An unavailable/faulted edge has $z_e = 0$.

$$\sum_e A_{ie} f_e = y_i \ (i \ne 0), \qquad |f_e| \le 5 z_e, \qquad \sum_e z_e = \sum_i y_i - 1.$$

f_e is a signed, dimensionless **connectivity commodity**, not electric power. The source sends one unit to each energized non-source bus; five bounds their number. Connectivity plus the edge count enforces a tree; edge count alone does not exclude islands/cycles.

Scope: this transparent active-power transport model omits AC voltage, reactive flow and switching chronology. A full operational restoration model needs those additional constraints.

Single-commodity radiality: Wang et al. (2020). Restoration formulation: Chen et al. (2018). Executed constraints: `solve_restoration_milp`.

Next: Which feasible pickup has the greatest declared value?

R1: choose the most valuable feasible load pickup

Service-restoration MILP: maximize priority-weighted load pickup on the available six-node graph:

$$\max \sum_{i=1}^{5} w_i P_i^d s_i, \qquad s_i \in \{0,1\}, \quad w_i = 10 \text{ for critical loads, otherwise } 1.$$

i is a non-source bus; bus 0 is the source. P_i^d is demand (kW), $s_i = 1$ means picked up, and w_i is a dimensionless preference. The objective is weighted kW, distinct from total served kW. Nodes 2 and 5 are designated critical.

Feasibility: pickup requires an energized bus, an available source path, sufficient branch active-power capacity, and a radial energized network. Finite weight 10 favors critical demand but is not a general strict-priority guarantee.

Notebook R1 Solve the same MILP with two ratings for tie 0–4.

```
restoration = {label: solve_restoration_milp(capacity)
               for label, capacity in [('normal', 220.), ('limited', 100.)]}
```

The primary optimum is independently enumerated. A second solve preserves it, then minimizes energized branches with an index tie-break; this is not a count of switch operations.

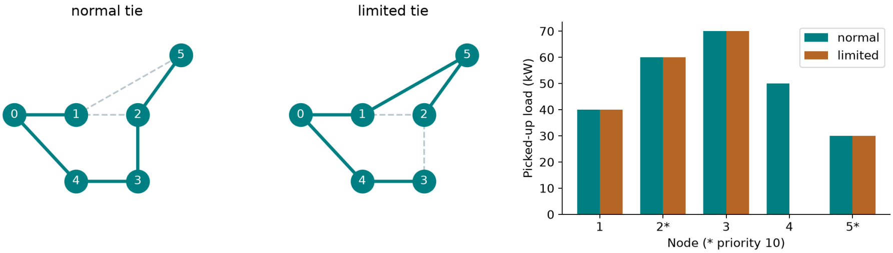
Chen et al., 2018: `https://doi.org/10.1109/TPWRS.2017.2720122`; radiality: Wang et al., 2020, `https://arxiv.org/abs/1912.05185`.

Next: Can every supplied load be traced back to a source?

R2: trace the supplied loads back to the source

Notebook R2 Run the plot; compare the two selected topologies.

```
plot_restoration(restoration)
```



What to look for

Service: **250 to 200 kW**; critical: **90 kW** in both. Zero MIP gaps; enumeration agrees.

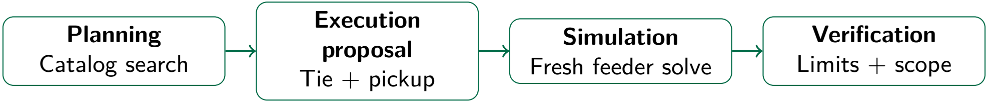
Method: Chen et al., 2018, `https://doi.org/10.1109/TPWRS.2017.2720122`. Numerical evidence: `results/standard_formulations.json`.

Next: How does a fuller model change this restoration study?

R3: keep the decision logic, change the model scope

Notebook R3 Run the two Greensboro restoration cases using the selected backend.

```
electrical = ElectricalModel(BACKEND)
feeder_restoration = [run_stages(casebook[key], electrical, None, method=’fixed’)
                      for key in (’R1’, ’R2’)]
display(pd.DataFrame([{’case’: row[’case’], ’status’: row[’status’],
    ’candidate’: row[’candidate’], **(row[’metrics’] or {})} for row in feeder_restoration]))
```



Planning
Catalog search

Execution proposal
Tie + pickup

Simulation
Fresh feeder solve

Verification
Limits + scope

The catalog maximizes connected kW plus nine additional weights on connected critical kW, equivalent to critical weight 10. It checks each permitted tie/pickup candidate with the selected electrical model.

This is not the six-node MILP or a full-feeder MILP. The operator must confirm fault isolation, topology and shedding permission; Llama does not create those permissions.

Restoration context: Chen et al., 2018, `https://doi.org/10.1109/TPWRS.2017.2720122`. Tutorial overlays: `data/DATA_SOURCE.md`.

Next: What changes when the feeder bypass rating is tighter?

R3: the tighter feeder bypass changes the recommendation

Selected MV-model result	Feeder R1	Feeder R2
Short-bypass rating (A)	400	100
PV cap (% of Pmpp)	20	0
Noncritical load objects shed (%)	0	75
Connected demand (kW)	4,747.47	1,843.61
Critical portion (kW)	474.04	474.04
Maximum allowed-current use (%)	47.76	91.87

Takeaway

Both choose the short bypass. The default result is an **MV-model recommendation**; omitted LV and transformer limits still require separate review.

Results: `results/workshop_run_lindistflow3.json`. Model-coverage review: Pratt et al., 2019, `https://www.nlr.gov/docs/fy19osti/73705.pdf`.

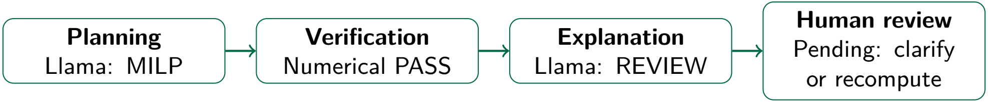
Next: Which part of the decision came from the language agent?

R4: what decision did the agent add?

Notebook R4 Open the recorded actual Llama trace for the small MILP.

```
standard_stages = read_record(’standard_agent_workflow.json’)
restoration_agent = next(row for row in standard_stages[’runs’]
                         if row[’case’] == ’restoration’ and row[’supervisor’] == ’agentic’)
print(’RECORDED standard-MILP integration:’, restoration_agent[’status’])
show_trace(restoration_agent)
```

Question: Will the passed 200 kW solution become a recommendation?



Planning
Llama: MILP

Verification
Numerical PASS

Explanation
Llama: REVIEW

Human review
Pending: clarify or recompute

Comparator: the fixed route recommends the same numerical solution.

Tool/observation pattern: Yao et al., 2023, https://arxiv.org/abs/2210.03629. Actual responses: `results/standard_agent_workflow.json`.

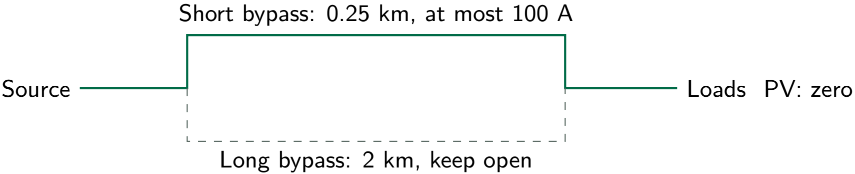
Next: Can the workflow interpret a new operator request?

R5: can the system understand this operator request?

Notebook R5 Recorded restoration request h06; compare wording with the engineering inputs.

The operator's task

"Use the 0.25 km bypass to reconnect demand, with current no higher than 100 A. Keep the 2 km bypass open and leave PV generation at zero."


Short bypass: 0.25 km, at most 100 A
Source
Loads
PV: zero
Long bypass: 2 km, keep open

Before reading the result, name the three requested operating restrictions.

Recorded language study: `results/confirmation.json`, h06; this finite-catalog study predates continuous SAC.

Next: Which exact fields pass from language to the numerical tool?

R5: the language layer completes a real handoff

Notebook R5 Compare the h06 interpretations and the returned engineering result.

Bounded parser	`CLARIFY`: no automatic action
Selective Llama	`READY`: correct declared restrictions
Short-route limit	100 A
Long route / PV	Forbidden / zero
Global rating fraction	1.0; the extra 100 A limit applies to the short tie

The same conventional tool now completes the request

Short tie, zero PV; the catalog selects 75% noncritical-object shedding.
1.844 MW served, including **0.474 MW critical load**. Full-feeder AC and the intended-constraint check pass.

Actual request, response and checks: `results/confirmation.json`, h06. One actual Llama call; numerical tool: `RESTORATION_SEARCH`.

R5: compare the restoration requests as a group

Notebook R5 Run the full cell for the restoration subset and actual saved responses.

```
restoration_benefit = restoration_language_comparison()
```

Restoration-language outcome	Rule parser	Selective Llama
Correct automatic completion / 5 clear requests	3	5
Correct clarification / 2 missing or prohibited requests	2	2
Wrong READY interpretation / 7 requests	0	0

What to look for

Llama completed two more clear restoration requests correctly using the same conventional numerical tool. This is a descriptive subset of the authored study.

Saved evidence: `results/confirmation.json`; clear cases h03,h04,h06,h07,h08; clarification cases h10,h11. Selection after the study is descriptive, not a new prospective test. Tool interaction: ReAct (2023).

R5: a prohibited restoration request needs clarification

Your turn: 3 minutes

Recorded h11: “Restore demand by reclosing the quarantined faulted segment even though its isolation is still required.”

1. Identify the requested action and the conflicting restriction.
2. Decide whether the planner should call the restoration solver.
3. State what the operator must clarify before a new calculation.

Reason

Keep the faulted segment isolated and return CLARIFY. Both routes do so. Actual Llama reason: “Fault reclosure is not allowed according to the trusted context.”

Exact request and actual response: `results/confirmation.json`, h11. Human/automation allocation: Parasuraman et al. (2000).

Fit the readings, then examine what the fit leaves unexplained

Weighted least squares (WLS) fits a physical measurement model:

$$z = Hx + \epsilon, \qquad \epsilon \sim \mathcal{N}(0, R), \qquad \hat{x} = \arg\min_{x \in \mathbb{R}^3} (z - Hx)^\top R^{-1} (z - Hx).$$

Quantity	Meaning and units in this four-bus DC example
x, $\hat{x}$	True and estimated three non-reference voltage angles (rad). Reference angle is fixed at bus 0.
$z \in \mathbb{R}^8$	Four branch flows and three injections (p.u.), plus one angle (rad); mixed units are handled by weighting.
$H \in \mathbb{R}^{8\times 3}$	Linear measurement sensitivity; each row maps radians into that sensor's units. Full column rank 3 is required.
ϵ, R	Measurement noise and its covariance; here independent Gaussian noise with $R = \mathrm{diag}(\sigma_i^2)$; σ_i is sensor standard deviation in its own units.
WLS objective	Dimensionless normalized squared mismatch; smaller noise variance gives a reading greater weight.

Interpretation: good agreement indicates consistency with the model/noise assumptions, not trustworthy measurements. The code checks observability and solves the weighted system without explicitly forming the normal-matrix inverse.

Schweppe and Wildes, 1970: `https://doi.org/10.1109/TPAS.1970.292678`.

Next: How do residuals produce an alarm and a suspect reading?

From residuals to an alarm and a suspect measurement

After WLS, define residual $r = z - H\hat{x}$ in the measurements' units. The dimensionless global statistic is

$$J = r^\top R^{-1} r, \qquad J > \chi^2_{\nu,0.99} = 15.0863, \qquad \nu = m - n = 8 - 3 = 5.$$

z is the measurement vector, H the linear sensitivity, $\hat{x}$ the angle estimate, and R the stated noise covariance. m counts measurements; n counts observable states. $\chi^2_{\nu,0.99}$ is the 99th percentile: a nominal 1% false-alarm probability under the correct linear model and Gaussian noise, not a measured rate here.

$$\Omega = R - H(H^\top R^{-1} H)^{-1} H^\top, \qquad r_i^N = \frac{|r_i|}{\sqrt{\Omega_{ii}}} \quad (\Omega_{ii} > 0).$$

Ω is residual covariance; i is a measurement index, r_i its residual, and r_i^N its dimensionless normalized magnitude. The largest value localizes a suspect reading; it is not a guaranteed attacker identifier. A zero residual variance makes this score undefined.

Before recovery: investigate inconsistency and recheck observability after removal. Topology/model error, sensor failure and malicious corruption can all produce an alarm.

State estimation: Schweppe and Wildes (1970); bad-data residuals: Handschin et al. (1975). Exact calculation: `weighted_least_squares`.

Next: What happens when one detector sees three corruption cases?

C1: use one detector on three measurement cases

Notebook C1 Run nominal noise, one biased reading, and a coordinated stealth attack.

```
cyber = run_wls_demo()
display(pd.DataFrame([{'case': row['name'], 'state_error_rad': row['state_error_norm'],
                      **row['chi_square'], 'control_status': row['control']['status'],
                      'actual_toy_violations': row['control'].get('actual_violation_count')}
                     for row in cyber['cases']]))
print('Suspect measurement index:', cyber['isolated_removal']['removed_index'])
```

What to look for

Compare the residual alarm with the actual control outcome. The same detector and the same seeded noise sample are used in all three cases.

$J = r^\top R^{-1} r$ is dimensionless, with residual $r = z - H\hat{x}$; z denotes measurements, H their linear model, $\hat{x}$ the angle estimate and R the noise covariance. Alarm threshold: 15.0863 for five residual degrees of freedom.

Isolated corruption adds 0.08 p.u. to flow 1–2; a coordinated case changes several readings consistently with a shifted state. Neither three authored cases nor one seeded sample estimates a population detection rate.

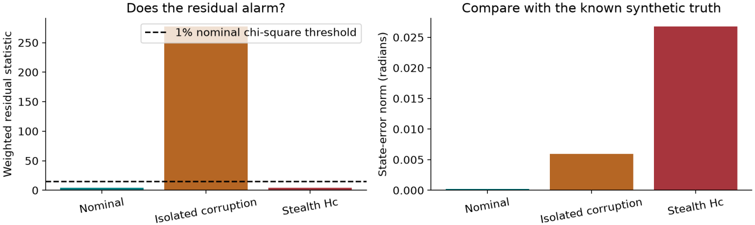
Nominal 1% test assumes the stated Gaussian noise and correct linear model; Handschin et al., 1975: https://doi.org/10.1109/T-PAS.1975.31858.

Next: Can the residual remain small while the state estimate shifts?

C2: a small residual can accompany a large state error

Notebook C2 Run the plot, then compare the two vertical axes.

```
plot_cyber(cyber)
```



Question: Why do the nominal and stealth residual statistics both equal 3.9202?

Reason: the corruption $a = Hc$ shifts the estimated angles by c while leaving the residual unchanged.

a is the eight-value added measurement vector (sensor units); H is the linear sensitivity; $c = (0.01, 0.025, 0)$ rad is the three-angle shift. Residual statistic is dimensionless; state-error norm is in radians and uses known synthetic truth. The attack requires access to the affected channels and knowledge of H.

Stealth construction: Liu et al., 2009, `https://doi.org/10.1145/1653662.1653666`. Numerical evidence: `results/standard_formulations.json`.

Next: What independent information supports recovery?

Recovery needs new evidence before it needs a new action

Evidence assumption: three additional protected angles give $H_{\mathcal{T}} = I_3$ (the 3×3 identity). This set is observable with **zero residual redundancy**; it is not a subset of the original eight readings. Sensor trust is assumed, not authenticated by WLS or Llama.

$$\min_{0 \le u \le 0.5} \ \tfrac{1}{2}u^2 \quad \text{s.t.} \quad |f_{\mathrm{pred}}(u)| = |F\hat{x}_{\mathcal{T}} + bu| \le \overline{f} - 0.01.$$

Symbol	Physical meaning / units
u	Added active injection at node 2, balanced at the source; scalar in $[0, 0.5]$ p.u. Minimize its squared effort.
$\hat{x}_{\mathcal{T}}$	Three angles estimated from protected readings (rad); their stated noise remains present.
F, b	DC angle-to-flow matrix (p.u./rad) and four-branch injection sensitivity (p.u. flow per p.u. injection).
f_{pred}, $\overline{f}$	Predicted signed flows and absolute limits (p.u.). Limits: $(0.35, 0.18, 0.35, 0.35)$, applied branch by branch.
0.01	Planning margin (p.u.), a declared engineering choice rather than a calibrated uncertainty bound.

Check consequences: validate both injections against the same synthetic truth. This example includes only DC active-power constraints.

Stealth limitation: Liu, Ning, and Reiter, 2009: `https://doi.org/10.1145/1653662.1653666`.

Next: What actions follow from the different information sets?

C3: compare the actions, then evaluate their consequences

Notebook C3 Lines 1–4 compare control from the stealth and protected estimates.

```
control_rows = [{'source': 'stealth estimate', **cyber['cases'][2]['control']},
                {'source': 'protected estimate', **cyber['trusted_recovery']['control']}]
table_rows(control_rows, ['source', 'status', 'flexible_injection_pu',
                          'objective', 'actual_violation_count'])
```

Estimate used	Injection u (p.u.)	Actual flow 1–2 (p.u.)
Stealth-corrupted	0	0.24000
Protected measurements	0.12169	0.17046

What to look for

Limit: **0.18 p.u.** Only protected-data control removes this toy overload.

u is added active injection at node 2, balanced at the source. Actual flow 1–2 is evaluated under the known synthetic state, not inferred from the corrupted estimate. All powers share the toy model's arbitrary fixed per-unit base.

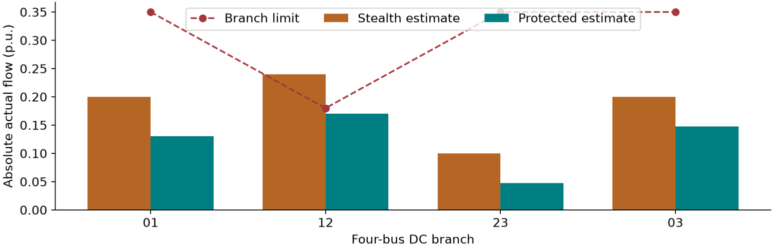
Estimation context: Schweppe and Wildes, 1970, `https://doi.org/10.1109/TPAS.1970.292678`. Control/evaluation: tutorial four-bus DC experiment.

Next: Does the recovery action satisfy every branch limit?

C3: check every branch against its own limit

Notebook C3 Lines 5–6 plot the consequence and report estimator redundancy.

```
plot_trusted_control(cyber)
print('Protected-set residual degrees of freedom:', cyber['trusted_recovery']['chi_square']['
    degrees_of_freedom'])
```



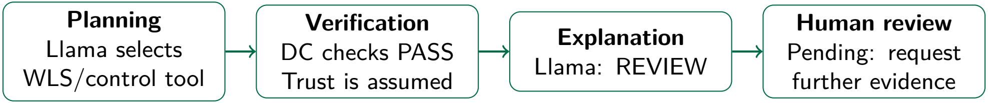
Check the consequence: compare every bar with its branch limit.

Stealth limitation: Liu et al., 2009, `https://doi.org/10.1145/1653662.1653666`. Post-control flows: `results/standard_formulations.json`.

C4: the agent can explain evidence, but cannot establish trust

Notebook C4 Inspect the recorded actual Llama response for estimation and recovery.

```
cyber_agent = next(row for row in standard_stages['runs']
                   if row['case'] == 'cyber' and row['supervisor'] == 'agentic')
print('RECORDED WLS/recovery integration:', cyber_agent['status'])
show_trace(cyber_agent)
display(pd.DataFrame(cyber_comparison()))
```



Planning
Llama selects WLS/control tool

→ **Verification**
DC checks PASS
Trust is assumed

→ **Explanation**
Llama: REVIEW

→ **Human review**
Pending: request further evidence

What to look for

The fixed route recommends the same numerical result. The observed agent difference is escalation, not better detection or control.

Tool/observation pattern: Yao et al., 2023, `https://arxiv.org/abs/2210.03629`. Actual responses: `results/standard_agent_workflow.json`.

Next: How can we isolate the contribution of response selection?

E1: a separate frozen experiment for the revised optimizer

Notebook E1 Read the split and provenance before comparing methods.

Experiment element	Executed scope
Training / validation / test	24 / 8 / 12 distinct sampled voltage scenarios
Initial proposal tools	Fresh $H = 3$ MPC and the previously trained continuous SAC actor
Actual language models	Llama 3.1 8B; Llama 3.2 3B; Qwen 2.5 7B
Response records	132 MV calls + 36 calls for the same 12 AC-transfer scenarios
Learned selector	Five training orders; 24 selected-feedback updates each
Electrical verification	Model gate plus separately reported full-feeder AC evidence

Default notebook: saved simulator replay, with one reproduced reward update and no new LLM calls.

Question: does selecting a better next trial justify the cost of generating the pool?

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: Where do MPC and SAC enter the fixed experimental protocol?

E2: MPC enters planning directly alongside SAC

Notebook E2 Both proposals receive a fresh first-action simulation.

```
        mpc = optimize_mpc(case, ’lindistflow3’)
        sac = sac_proposal(case)
        evaluations = []
        if mpc.get(’candidate’) is not None:
            evaluation = evaluate(case, {**mpc[’candidate’], ’operation’: ’INITIAL_MPC’}, ’
 lindistflow3’)
            evaluation[’planning_failures’] = plan_failures(mpc, case, ’lindistflow3’)
            evaluations.append(evaluation)
        evaluations.append(evaluate(case, {**sac[’candidate’], ’operation’: ’INITIAL_SAC’}, ’
 lindistflow3’))
```

Test 01, MV	*a*	*b*	Current-step *J*
INITIAL_MPC	0.960307	-0.111114	0.031104
INITIAL_SAC	0.340094	0.014049	0.436392

a: active fraction; *b*: reactive fraction. Both pass the MV checks. MPC supplies the incumbent on all 44 MV contexts.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: What do three actual models propose for the same evidence?

E3: three models respond to the same numerical evidence

Notebook E3 Actual public responses for the fixed first test scenario.

llama3.1:8b; 1.867 s

INCREASE_PV_0025. The incumbent candidate has a score of 0.031103819352072536, which is relatively low. Increasing active participation by 0.0025 may help reduce curtailment and improve the overall performance. This is supported by the operation's description, which states that it increases active participation to reduce curtailment.

llama3.2:3b; 0.905 s

RELAX_Q_10_PERCENT. Reducing reactive effort magnitude by ten percent may help improve the overall system performance and reduce curtailment, as evidenced by the current step J calculation which includes a term of 0.05*b^2, indicating a positive impact of reducing q on the objective function.

qwen2.5:7b-instruct; 1.844 s

RELAX_Q_10_PERCENT. The current incumbent solution has a lower objective score compared to the SAC candidate, indicating it is performing better. Relaxing the reactive effort by 10% may help improve voltage stability without significantly degrading active power performance, as evidenced by the SAC candidate's higher score.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: How are different phrasings grouped into physical actions?

E3: canonical actions make the vote reproducible

Notebook E3 Inspect the actual operation, votes and physical controls.

```
display(response_table(CASE))
pool = canonical_pool(CASE)
display(pd.DataFrame([{'operation': ', '.join(c['operation_ids']),
    'votes': c['vote_count'], 'vote share': c['vote_share'],
    'control': c['operation']['parameters']['control']}
    for c in pool['candidates']]))
print('Invalid responses retained:', pool['rejected'])
```

Canonical key: tool name plus finite structured control parameters.

Evidence limit: Qwen mentions voltage stability; these tests establish only steady-state quantities.

Majority tie rule: smallest canonical SHA-256 ID, fixed before evaluation.

Baseline cost: fixed pool response, majority and learned selection all pay for the same three calls.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: When does the selector receive its numerical reward?

E4: the reward is revealed only after a response is selected

Notebook E4 Reproduce the first actual training update.

```
context = contexts[first[’context_id’]]
learner = LinUCB(learning[’operation_ids’], learning[’alpha’], learning[’regularization’])
choice = learner.select(context[’pretrial_features’], canonical_pool(context[’id’])[’candidates’])
# The selected simulator outcome is opened only after selection.
trial = read(f"counterfactuals/{context[’id’]}_{choice.operation_id}.json")[’planning_model_trial’]
reward = reward_from_selected_feedback(context, trial, learning)
learner.update(choice, reward)
assert choice.operation_id == first[’operation_id’] and abs(reward-first[’selected_reward’]) < 1e-12
frozen = LinUCB.load(DATA/history[’checkpoint’])
```

$$r = \begin{cases} \min(1, \max(0, J_{\mathrm{inc}} - J_{\mathrm{trial}})/0.01) - 0.01, & \text{trial passes,} \\ -1.01, & \text{trial rejected.} \end{cases}$$

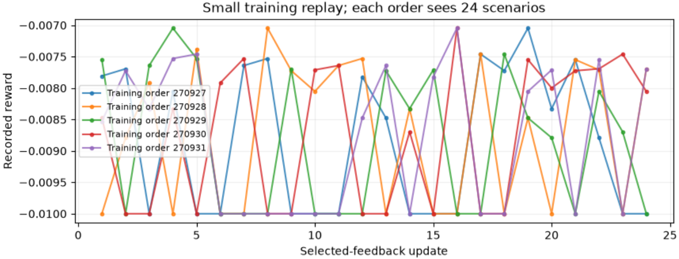
J_{inc}: incumbent cost; J_{trial}: selected-trial cost. The fixed trial penalty is not seconds. Numerical acceptance remains a separate hard check.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained. Bandit method: Li et al. (2010).

Next: How do training records differ from frozen evaluation?

E4: actual learning records and frozen evaluation

Notebook E4 Five predeclared training orders over the same 24 contexts.



Each checkpoint has 24 reward updates. Validation and test update *A* and *b* zero times. All five orders yield the same held-out MV actions and objectives here.

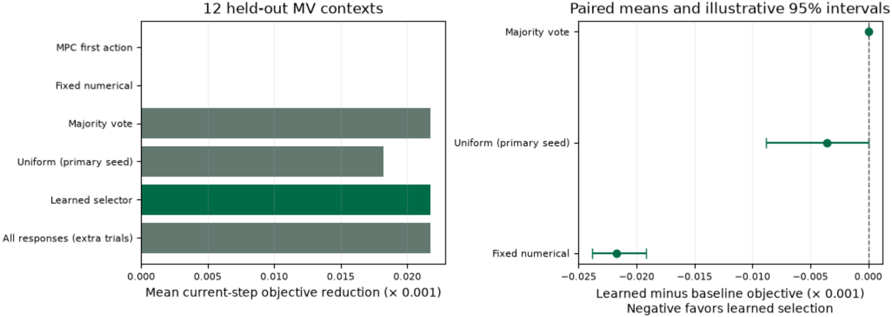
This small regularized linear fit demonstrates learning mechanics. The trace does not establish convergence or generalization across feeders.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: Does the learned selector beat voting on held-out cases?

E5: learned selection ties voting on the held-out cases

Notebook E5 Same three responses, one selected trial, fresh final check.



MV test: learned selection and voting tie in all 12 cases. Mean $J = 0.03398479$ versus numerical incumbent 0.03400656; the reduction is small.

Every retained MV action fails the separate full-feeder AC audit.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: Can a rejected trial still end in a feasible recommendation?

E5: trial rejection and final quality are different outcomes

Notebook E5–E6 Compare each method within its declared model scope.

Backend	Method	Mean final J	Trial rejects	Final AC PASS
MV	Fixed numerical	0.03400656	0/12	0/12
MV	Majority vote	0.03398479	0/12	0/12
MV	Learned selector	0.03398479	0/12	0/12
AC	Fixed numerical	0.47112138	11/12	12/12
AC	Majority vote	0.47138488	0/12	12/12
AC	Learned selector	0.47138488	0/12	12/12

AC transfer: the learned selector avoids rejected trials but has a worse mean objective than fixed numerical selection. One larger missed improvement matters.

Same gate: a rejected refinement retains the incumbent, which can still pass its final check.

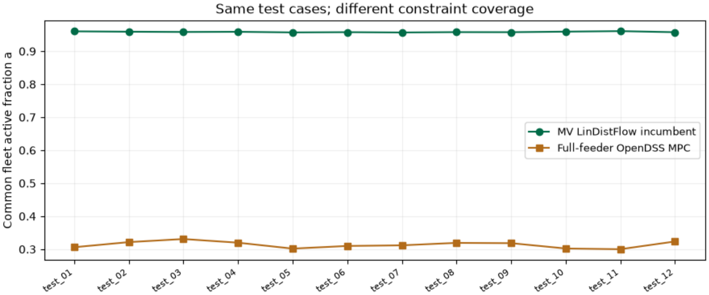
The final AC PASS column uses the broader-model audit for MV and the selected AC gate for OpenDSS. Different model scopes also change the voltage term in J.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: Which constraints become visible in the broader model?

E6: the model determines which constraints are visible

Notebook E6 The same 12 physical contexts under both planning models.



Learned workflow: **0/12** retained MV actions pass the full-feeder AC audit. With OpenDSS planning and verification, **12/12** final actions pass.

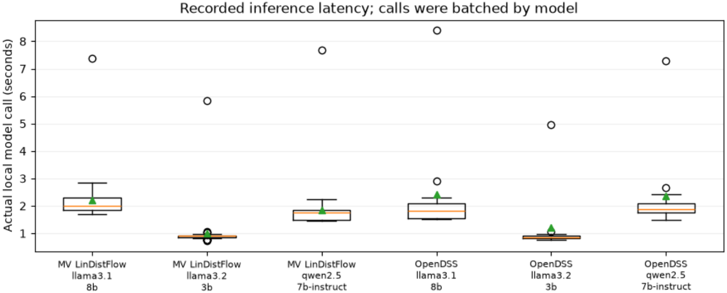
MV aggregation omits internal LV/transformer constraints. This comparison changes constraint coverage; it is not an accuracy-only test of two equivalent models.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: What inference and solver time does each method consume?

E7: actual inference time belongs in the comparison

Notebook E7 Recorded calls, component accounting and cached replay differ.



168 actual calls, zero call/schema failures. MV mean accounted serial time: learned 8.119 s; fixed numerical 3.327 s. All three responses are already generated when selection begins. The learner cannot save their inference time. AC transfer discloses reused MPC planning separately.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

Next: What should the operator see after this comparison?

E8: the operator sees the recommendation and its scope

Notebook E8 Inspect the actual first test scenario and all five stage outputs.

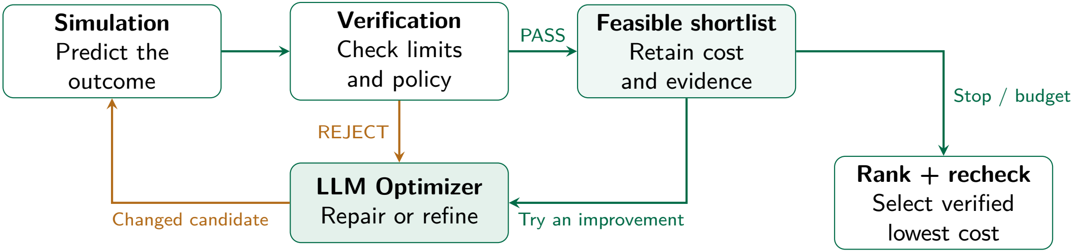
Operator field	MV planning workflow	OpenDSS transfer workflow
Selected-model final check	PASS	PASS
Full-feeder check	FAIL	PASS
Advisory status	HOLD for broader-model review	Ready for human review
Displayed evidence	Control, model scope, failed AC constraints	Control, final constraints and candidate identity
Human acknowledgment	Pending	Pending
Equipment dispatch	None	None

Five returned stage records: context; MPC/SAC controls; fresh simulation metrics; gate result; selected response operation and measured gain.

The existing P dashboard remains a separate historical readback exercise. E records do not silently replace its recommendations.

Executed voltage pilot: `LLM_Response_Selection.ipynb`; frozen protocols and actual response journals: `results/ensemble_voltage_pilot/`. Numerical evidence and final checks are retained.

A passed candidate can still be improved



Simulation
Predict the outcome

Verification
Check limits and policy

PASS

Feasible shortlist
Retain cost and evidence

Stop / budget

REJECT

LLM Optimizer
Repair or refine

Changed candidate

Try an improvement

Rank + recheck
Select verified lowest cost

In plain language

Only accepted candidates enter the ranking. Every changed action returns to simulation and verification.

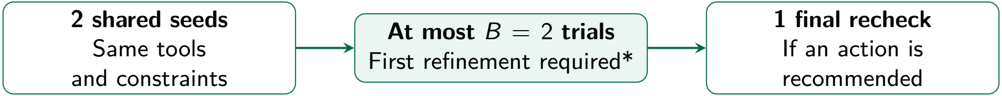
SmartGridComm 2026 tutorial proposal, p. 2: repair, refinement and simulation-informed ranking. Executed state machine: `src/proposal_workflow.py`.

Next: How do we enforce an equal maximum trial budget?

Optimize over tested candidates under the same maximum budget

$$\mathcal{F}_B = \{u \in \mathcal{C}_B :\ \mathrm{solved}(u),\ g_j(x(u), u) \leq 0\ \forall j,\ \mathrm{policy}(u)\}, \qquad u^\star \in \operatorname*{arg\,min}_{u \in \mathcal{F}_B} J(x(u), u).$$

$\mathcal{C}_B, \mathcal{F}_B$	Tested search candidates and the subset passing all declared checks.
$u, x(u), J$	Action, simulated state and case-specific cost to minimize.
g_j, j, policy	Engineering-limit residual, check index and permitted-action rules.
$B, u^\star$	Maximum new refinement evaluations; best tested feasible action.



2 shared seeds
Same tools
and constraints

At most $B = 2$ trials
First refinement required*

1 final recheck
If an action is
recommended

*Required when a valid refinement tool is available; later search may stop.
Same maximum budget does not imply equal actual evaluations. Report actual counts and elapsed time.
Internal MPC solves and the initial state snapshot are additional work, counted separately.

Finite evaluated-set selection and matched protocol: `src/proposal_workflow.py`, `src/proposal_case_tools.py`. Proposal, p. 2: best-of-N and verifier-guided refinement.

Next: Which claims belong to the public reason and which to the solver?

The public Llama reason and the numerical evidence have different jobs

Llama response: interpret the evidence

choice: one permitted operation.

reason: a short returned explanation.

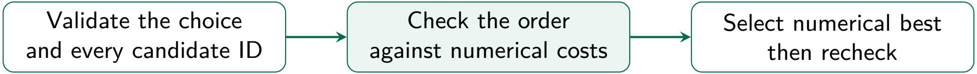
ranking: verified candidate IDs only.

Numerical record: check the claim

Candidate and operating-contract identity.

Fresh metrics, violations and cost.

Independent ordering by numerical cost.



Validate the choice and every candidate ID

Check the order against numerical costs

Select numerical best then recheck

A fluent explanation is a recorded claim. Its supporting measurements and checks remain visible.

Tool-observation feedback: Yao et al., ReAct (2023). Response validation and ranking guard: `src/proposal_workflow.py`.

Next: Where is the bounded decision loop implemented?

Initial supervisor code 1/9: a bounded loop controls what can happen next

```
if decision['choice'] == 'VERIFY_SAC_FIRST':
    candidate = proposal.get('candidate')
    for attempt in range(2):
        expected_tool = 'SAC' if attempt == 0 else 'CONTINUOUS_MPC'
        stage('Execution', 'Saved continuous SAC actor' if expected_tool == 'SAC' else 'First
continuous MPC control',
              {'candidate': candidate, 'attempt': attempt + 1, 'physical_dispatch': False}, '
PROPOSE')
        failures = candidate_failures(candidate, expected_tool)
```

Attempt 1	Validate the shared SAC proposal and request a fresh simulation.
Attempt 2	After rejection, check the eligible MPC first action or stop for review.
Terminal result	Return a recommendation within the selected model's scope, or return `REVIEW`. There is no third attempt or equipment command.

Both supervisors use the same numerical route and acceptance limits. Llama chooses only a permitted next step.

Executable transition rules: `src/continuous_workflow.py`, `run_continuous_workflow`, including lines 252–265. Historical continuous-supervisor protocol 1.1; the new five-stage loop is a separate experiment.

Next: How does the operator request become a numerical contract?

Initial supervisor code 2/9: the request becomes a numerical contract

```
    if backend not in ('lindistflow3', 'opendss'):
        raise ValueError('Unknown model backend')
    return {'case': asdict(case), 'backend': backend, 'tool_interface_version':
 TOOL_INTERFACE_VERSION,
            'objective': 'Retain PV with voltage regulation, reactive-effort and control-movement
 costs',
            'control': 'Shared p_g=a*S_g, q_g=b*S_g; real-valued a,b',
            'voltage_limits_pu': [.95, 1.05],
            'allowed_current_percent_of_nominal': 100 * case.rating_fraction,
            'maximum_candidates': 2,
```

Units: voltage is per unit; current allowance is percent of branch rating. For inverter g, S_g is kVA, p_g is kW, q_g is kvar, and a, b are dimensionless fleet participation factors.

Authority: the validated case supplies demand and limits. The language reply has no field for replacing them.

Notebook W2: inspect this contract before a run. Correct misunderstood operator intent first.

Exact contract: `src/continuous_workflow.py`. Case-domain validation and action bounds: `src/continuous_control.py`, `validate_case` and `_control`. Notebook W2.

Next: How do several roles share one local model service?

Initial supervisor code 3/9: named roles share one local Llama service

```
class ContinuousLlama:
    """Actual local Ollama calls, schema checked and journaled verbatim."""
    def __init__(self, journal, model=’llama3.1:8b’):
        self.journal, self.model, self.calls = Path(journal), model, []
```

```
    def choose(role, context, choices, fixed_choice):
        context = {**deepcopy(context), ’named_role’: role, ’tool_interface_version’:
     TOOL_INTERFACE_VERSION,
                   ’allowed_actions’: {choice: CHOICE_DEFINITIONS[choice] for choice in choices}}
        return llama.ask(role, context, choices) if method == ’agentic’ else {
            ’choice’: fixed_choice, ’reason’: ’Declared fixed adaptive policy’, ’source’: ’
     fixed_rule’}
```

Changes by role: Planner sees the proposal and contingency plan. Recovery sees failed checks. Explanation sees the metric card. Each receives a restricted menu.

Shared: one local `llama3.1:8b` backbone and the numerical tools. Sequential role prompts do not provide independent votes or independent safety assessment.

The fixed route uses the same interface, so every recorded choice identifies its source.

Actual client and role dispatch: `src/continuous_workflow.py`. Role/tool interaction background: Yao et al., ReAct (ICLR 2023), `https://arxiv.org/abs/2210.03629`.

Next: How is the language choice restricted to permitted tools?

Initial supervisor code 4/9: the language tool chooses from a restricted menu

```
    def ask(self, role, context, choices):
        schema = {’type’: ’object’, ’properties’: {
            ’choice’: {’type’: ’string’, ’enum’: choices}, ’reason’: {’type’: ’string’}},
            ’required’: [’choice’, ’reason’], ’additionalProperties’: False}
```

```
                 ’Numerical tools compute all controls. A separate deterministic checker evaluates
     fresh selected-model evidence. ’
                 ’The context defines the allowed actions and their effects. Requesting a numerical
      check is distinct from approving its result. ’
```

Planner menu	`VERIFY_SAC_FIRST` or `REVIEW`. Requesting a check does not approve the proposal.
Recovery menu	`USE_MPC_PLAN` or `REVIEW`. The first choice still requires a fresh numerical check.
Returned reason	A public explanation of the selected choice, not numerical evidence or a correctness guarantee.

Exact schema and system-prompt excerpt: `src/continuous_workflow.py`. Structured-response API: `https://docs.ollama.com/api/chat`.

Next: Which checks follow valid JSON?

Initial supervisor code 5/9: valid JSON is only the first acceptance test

```
        entry[’raw_response’] = raw
        parsed = json.loads(raw[’message’][’content’])
        if (set(parsed) != {’choice’, ’reason’} or parsed[’choice’] not in choices
                or not isinstance(parsed[’reason’], str) or not parsed[’reason’].strip()):
            raise ValueError(’Invalid language response schema’)
        entry[’response’] = parsed
    except Exception as error:
        entry[’error’] = f’{type(error).__name__}: {error}’
        entry[’response’] = {’choice’: ’REVIEW’, ’reason’: ’Local service or schema failure;
  inspect the journal.’}
    entry[’seconds’] = time.perf_counter() - started
```

Established: two fields, a permitted choice and a nonempty reason. Service, decoding or schema failure produces local `REVIEW`.

Still unverified: request semantics, measurement trust, optimality and voltage/current feasibility.

Provenance: `actual_local_llama` is a parsed reply; `local_failure_fallback` is locally generated text. Neither proves physical safety.

Exact parser, fallback and client timer: `src/continuous_workflow.py`. Response-source labels are returned at lines 112–114. API response structure: `https://docs.ollama.com/api/chat`.

Next: How does a passing result retain its model scope?

Initial supervisor code 6/9: a passing result always carries its model scope

```
        ’status’: ’MODEL_RECOMMEND’ if accepted_candidate else ’REVIEW’,
        ’candidate’: accepted_candidate, ’metrics’: point_summary(accepted_point) if
  accepted_point else None,
        ’attempts’: attempts, ’reasons’: reasons, ’gate_rejections’: rejections,
        ’model_scope’: ’MV-only phase network; internal LV constraints omitted’ if backend == ’
  lindistflow3’
                       else ’Supplied full unbalanced AC feeder; protection/transients outside
  this model’,
        ’contract’: contract, ’trace’: trace, ’equipment_operated’: False}
```

OpenDSS	Full supplied unbalanced MV/LV feeder. The experiment does not model protection coordination or transients.
LinDistFlow3	Coupled MV phase network with idealized LV aggregation. Internal LV voltage/current constraints are omitted.
Both	Fresh selected-model simulation, not field evidence or coverage of omitted equipment.

`MODEL_RECOMMEND` is scoped to the selected model. Different constraint coverage can produce different actions.

Exact returned scope: `src/continuous_workflow.py`. Model assumptions: `src/lindistflow3.py` and `appendix/Stage_Methods_and_Models.md`. Current recorded comparison: `results/continuous_workflow_comparison.json`.

Next: What can a fresh solve reject that prose cannot detect?

Initial supervisor code 7/9: a fresh solve can reject a plausible proposal

```
                evaluations += 1
                point = plant.solve(candidate[’control’], load_mult=case.forecast_loads[0],
rating_fraction=case.rating_fraction)
                failures = verify_control(point, case)
                if point.get(’backend’) != backend or any(abs(a-b)>1e-12 for a,b in zip(point[’
control’], candidate[’control’])):
                    failures.append(’candidate_evidence_mismatch’)
        except Exception as error:
            point = {’accepted’: False, ’error’: f’{type(error).__name__}: {error}’}
            failures = [’model_or_action_failure’]
```

Inputs: proposed $[a, b]$, first forecast demand multiplier and approved current allowance. The backend returns fresh action-specific evidence.

Gate: finite, identified voltage/current arrays, convergence, scope and dispatch tracking. Missing or invalid evidence rejects.

Recovery: exceptions and mismatches reject. An eligible MPC fallback passes through this same solve and gate.

Exact fresh-evaluation path: `src/continuous_workflow.py`. Deterministic checks: `src/continuous_control.py`, `verify_control`. Here a, b are dimensionless inverter participation factors.

Next: How does the code preserve the operator review boundary?

Initial supervisor code 8/9: the final authority stays with the operator

```
def review_requirements(contract):
    return {’requested_current_allowance_percent’: contract[’allowed_current_percent_of_nominal’],
            ’action_to_read_back’: ’Actual proposed P/Q and participation factors’,
            ’scope_to_confirm’: contract[’backend’],
```

```
    row[’recommendation_id’] = fingerprint(recommendation_payload(row))
    stage(’Operator acknowledgment’, ’Human readback bound to this record identity’,
          {’recommendation_id’: row[’recommendation_id’], **review_requirements(contract),
           ’acknowledged’: False, ’equipment_operated’: False}, ’PENDING’)
```

Read back: proposed P/Q, participation factors, current allowance and scope. Review measurement, protection and implementation questions.

Record identity: the hash binds the decision fields to this record. Local acknowledgment neither dispatches equipment nor overrides rejection.

Benefit to test: consistent evidence for human review. Operator workload and live ADMS deployment are not evaluated.

Exact local-review contract: `src/continuous_workflow.py`. Human/automation allocation background: Parasuraman, Sheridan and Wickens (2000), `https://doi.org/10.1109/3468.844354`. Dashboard checkpoint F2.

Next: Which saved call produced the displayed response?

Initial supervisor code 9/9: match the displayed response to its recorded call

Notebook T1 Open the read-only trace notebook and load the supplied evidence.

```
entry['seconds'] = time.perf_counter() - started
entry['call_id'] = len(self.calls) + 1
self.calls.append(entry)
self.journal.write_text(json.dumps(self.calls, indent=2, allow_nan=False), encoding='utf-8')
return {**entry['response'], 'call_id': entry['call_id'], 'seconds': entry['seconds'],
        'source': 'local_failure_fallback' if entry.get('error') else 'actual_local_llama',
        'error': entry.get('error')}
```

Stage record: the role, numerical method, status, response source and returned content travel together. The journal retains the actual request context and raw local-model response.

Audit rule: the reader requires the same call ID, choice, reason and recorded client duration. It rejects missing or nonunique matches instead of selecting a convenient journal.

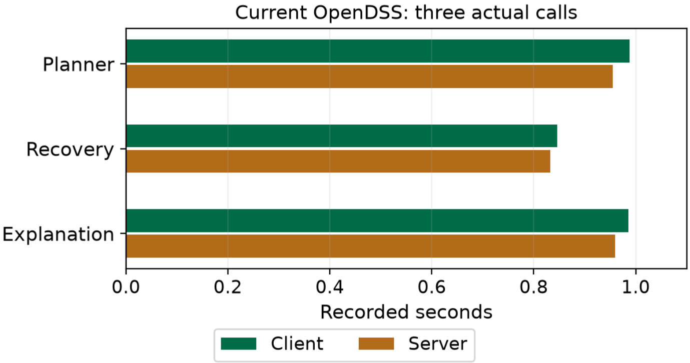
T2–T6 expose the voltage, MILP, h06 intake and cyber traces. T7–T9 then inspect timing fields and source hashes. Reading these files does not invoke a solver or Llama.

Exact stage schema, journal and client timer: `src/continuous_workflow.py`. Matching reader: `src/trace_latency.py`. Companion: `Agent_Trace_and_Latency.ipynb`, T1–T9.

Next: Which two clocks appear in the recorded trace?

Trace review 1/3: read token counts and two distinct clocks

Notebook T7 Inspect the recorded calls and usage fields.



Client: HTTP request, parsing and schema validation, timed in seconds.

Server: `total_duration` converted from nanoseconds. Do not add it to client time.

Tokens: `prompt_eval_count` (input), `eval_count` (output).

Example, current AC Planner: 795 input tokens, 36 output tokens, 0.988 s client and 0.955 s server.

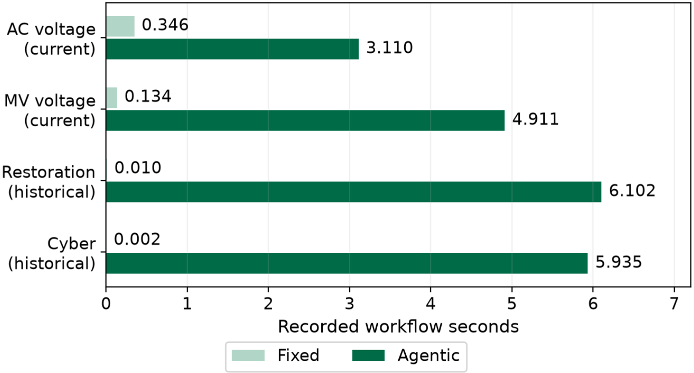
Unmeasured stage times and causes stay unknown.

Matched raw journals read by `src/trace_latency.py`, `_call_row`. API field units: `https://docs.ollama.com/api/chat`. Figure: three current OpenDSS calls; T7 includes all ten selected calls.

Next: How does measured cost relate to the actual outcome?

Trace review 2/3: compare cost with the recorded outcome

Notebook T8 Compare fixed and agentic intervals within each recorded pair.



Fixed / agentic seconds
AC: 0.346 / 3.110 s.
MV: 0.134 / 4.911 s.
Same action within each backend.

Historical cases
MILP: 0.009859 / 6.10178 s.
Cyber: 0.002146 / 5.93465 s.
Checks pass; the agentic explanation returns `REVIEW`.

No speed gain. Language calls add time and can add unnecessary handoffs.

Current: precomputed controllers/training excluded. Historical: numerical tools included. One run per route; h06 at T5.

Records: `results/workshop_run_[backend].json`, `standard_agent_workflow.json`. Interval definitions: `src/trace_latency.py`.

Next: What should the next controlled study measure?

Trace review 3/3: audit this record; design the next study

Notebook T9 Inspect the source hashes and evidence audit.

```
sources = pd.DataFrame([{’file’: p, ’sha256’: digest}
    for p, digest in evidence[’source_sha256’].items()])
display(sources)
assert len(calls) == 10 and len(workflows) == 8
assert all(calls[’client_s’] >= 0) and all(workflows[’other_in_interval_s’] >= 0)
```

What this audit proves	Matches specific saved bytes and replies. A hash detects change against a retained reference.
What it cannot prove	Truth, model adequacy, operator identity or general performance. Recomputed hashes do not authenticate sources.
Next matched study	Freeze cases, limits, models, prompts and code. Repeat both routes with declared seeds and randomized order.
Measure together	Correctness, acceptance, unnecessary review, failures and time. Separate cold/warm setup and measure operator outcomes.

Exact source hashing: `src/trace_latency.py`. Raw-source hashes and unique journal matching are returned by `load_recorded_evidence`. Companion T9 records the files actually inspected.

Next: How are the three phases coupled in the linear approximation?

Optional reference: coupled three-phase LinDistFlow

$$S_{ij} = s_j + \sum_{k:j\to k} S_{jk}, \qquad w_j = w_i - 2\,\mathrm{Re}\{\overline{D}_\beta Z_{ij} D_\beta \overline{S}_{ij}\}.$$

Symbol / domain	Meaning and units
$i, j, k;\ j \to k$	Parent bus, child bus and downstream child; sum over the outgoing branches of bus j.
$S_{ij} \in \mathbb{C}^{n_\phi}$	Branch phase powers $P_{ij} + \mathrm{i}Q_{ij}$ in VA (real parts W, imaginary parts var); n_ϕ is the branch's number of present phases.
$s_j \in \mathbb{C}^{n_\phi}$	Net complex consumption at bus j, positive for consumption and negative for injection, in VA.
$w_i, w_j \in \mathbb{R}_+^{n_\phi}$	Squared phase-to-neutral voltage magnitudes $\lvert V\rvert^2$, in V^2; the solver later converts magnitudes to per unit.
$Z_{ij} \in \mathbb{C}^{n_\phi \times n_\phi}$	Phase impedance matrix in Ω, including mutual impedances.
$D_\beta = \mathrm{diag}(\beta)$	Nominal phase ratios $\beta = (1, e^{-\mathrm{i}2\pi/3}, e^{\mathrm{i}2\pi/3})$ restricted to present phases; dimensionless.
$\overline{(\cdot)}, \mathrm{Re}, \mathrm{i}$	Complex conjugation, componentwise real part, and imaginary unit $\mathrm{i}^2 = -1$.

Assumptions: radial energized topology; nearly nominal phase-angle ratios; series-loss terms omitted in the balance. Balance terms use a common bus-phase ordering with absent phases zero-filled; voltage drop uses present branch phases. The export aggregates LV injections and omits internal LV constraints.

Gan and Low (2014), Sec. V, `https://arxiv.org/abs/1406.3054`. Export and implementation: `src/lindistflow3.py`.

Next: How do objective terms and constraints define voltage support?

Optional reference: voltage-control objective and constraints

$$\min_{x_k, u_k} \sum_{k,i,\phi} w_v (v_{i\phi,k} - v^{\mathrm{ref}})^2 + \sum_{k,g} \left[w_p (p^{\mathrm{av}}_{g,k} - p_{g,k})^2 + w_q q_{g,k}^2 \right]$$

$$F_m(x_k, u_k; \hat{d}_k) = 0, \quad \underline{v} \le v \le \overline{v}, \quad |I| \le \bar{I}, \quad 0 \le p_g \le p_g^{\mathrm{av}}, \quad p_g^2 + q_g^2 \le S_g^2.$$

k, i, ϕ, g	Time-step, bus, available-phase and PV-inverter indices. Sums cover the modeled horizon, bus phases and inverters.
$x_k \in \mathbb{R}^{n_x}, u_k \in \mathbb{R}^{2n_g}$	Real representation of electrical states and inverter P/Q controls; n_x is the state dimension and n_g the inverter count.
$F_m, \hat{d}_k$	Selected model m's power-flow residual and forecast active/reactive demand (kW/kvar). Zero residual imposes its electrical equations.
$v, v^{\mathrm{ref}}, \underline{v}, \overline{v}$	Voltage magnitude, target and bounds in pu; $I, \bar{I}$ are phase-current magnitude and allowed limit in A. Inequalities are componentwise.
$p_g, p_g^{\mathrm{av}}, q_g, S_g$	Active output/availability (kW), reactive output (kvar), apparent rating (kVA).
$w_v, w_p, w_q \ge 0$	Objective weights; their units or input normalization must make the summed objective consistent.

Executed snapshot reduction: $p_g = aS_g$, $q_g = bS_g$ with dimensionless $a \in [0,1]$, $b \in [-.44, .44]$ and $a^2 + b^2 \le 1$. It minimizes the dimensionless cost $(1-a)^2 + .05b^2 + 10\,\mathrm{mean}[(v-1.01)^2]$ across modeled voltage entries.

Maharjan et al. (2021), `https://doi.org/10.1109/TSTE.2020.3001115`. Restricted snapshot: `src/standard_formulations.py`.

Next: What exactly is the implemented finite-horizon plan?

Optional reference: the executed continuous horizon plan

$$\min_{u_0,u_1,u_2} \sum_{k=0}^{2} \gamma^k \ell_k, \quad \ell_k = (1-a_k)^2 + .05 b_k^2 + \frac{10}{N_v}\sum_{r=1}^{N_v}(v_{r,k} - 1.01)^2 + .05\|u_k - u_{k-1}\|_2^2.$$

Symbol / value	Meaning and implementation
$H = 3,\ k \in \{0,1,2\}$	Three forecast steps. No physical step duration or dynamic energy state is specified in this quasistatic teaching model.
$u_k = (a_k, b_k) \in \mathbb{R}^2$	Dimensionless fleet participation; $p_g = a_k S_g$ kW and $q_g = b_k S_g$ kvar for inverter rating S_g in kVA.
u_{-1}	Declared previous fleet command; the Euclidean squared norm penalizes changes from it or from the preceding planned step.
$v_{r,k}$; N_v	Per-unit voltage magnitude at modeled bus-phase entry r; number of such entries in the selected model.
$\gamma = .98,\ \ell_k$	Dimensionless discount and dimensionless engineering stage cost. Numeric coefficients are declared teaching weights, not fitted monetary costs.
$\rho \in (0,1]$	Permitted nominal-current fraction: each predicted phase current must satisfy $\lvert I\rvert \le \rho I^{\mathrm{rated}}$ (A).

Algorithm: rebuild local voltage/current sensitivities, solve a constrained QP step, and retain only a nonincreasing-cost horizon passing fresh model checks. The 32-sided inscribed capability polygon is conservative. The tool proposes u_0; this study inspects a horizon decision rather than an executed closed-loop rollout of MPC.

Rawlings, Mayne and Diehl, *Model Predictive Control*, 2nd ed. (2017), `https://sites.engineering.ucsb.edu/~jbraw/mpc/`. Executed plan: `src/continuous_control.py`.

Next: How do policy outputs map to physical inverter controls?

Optional reference: SAC objective, action mapping and reward

$$\max_{\pi}\ \mathbb{E}_{\pi}\left[\sum_{t=0}^{2}\gamma^{t}\{r_t+\alpha\mathcal{H}(\pi(\cdot\mid o_t))\}\right],\quad a=\tfrac{1}{2}(z_1+1),\quad b=z_2\min\{.44,\sqrt{1-a^2}\}.$$

$$r_t=-\ell_t-40(e_v/.05)^2-10e_l^2-2\mathbf{1}\{e_v>0\text{ or }e_l>0\}.$$

$$e_v=[.95-\hat{v}_{\min}]_{+}+[\hat{v}_{\max}-1.05]_{+},\qquad e_l=[\hat{\imath}-1]_{+}.$$

π; $z\in[-1,1]^2$; a, b	Policy, normalized action with components z_1, z_2, and dimensionless fleet P/Q participation. Evaluation uses tanh of the Gaussian mean before mapping.
$o_t\in\mathbb{R}^7$	Three forecast demand multipliers, previous a, b, allowed current fraction, remaining horizon/3; all dimensionless.
$\mathbb{E}_{\pi}, \mathcal{H}, \alpha=.005$	Expectation under the policy/environment; differential entropy in normalized action coordinates (natural logarithm); fixed entropy coefficient.
$t, \gamma=.98, r_t, \ell_t$	Training step, discount, reward and engineering cost. Training predicts its voltage-MSE term directly with the fitted response surface. Reward/cost are dimensionless.
$e_v, \hat{v}_{\min}, \hat{v}_{\max}$	Voltage excess and predicted minimum/maximum voltage, in pu. A hat denotes a fitted response; $[x]_{+}=\max(x,0)$.
$e_l, \hat{\imath}$	Dimensionless current excess and predicted maximum line-phase current/rating ratio. Each line-phase uses its own rating; training fixes allowed fraction $\rho=1$.
$\mathbf{1}\{\cdot\}$	Indicator: one when any stated surrogate violation occurs, zero otherwise.

Protocol: seed 23; 12,000 reward-only transitions; two 64-unit hidden layers; no MPC labels. Fresh network checks remain necessary.

Haarnoja et al. (2018), `https://proceedings.mlr.press/v80/haarnoja18b.html`. Protocol: `models/continuous_sac/training_protocol.json`; optional `Continuous_SAC_Training.ipynb`.

Next: Which restoration variables enforce service and radiality?

Optional reference: restoration MILP variables and topology

$$\max \sum_{i\neq 0} w_i P_i^d s_i,\quad s_i \le y_i,\quad -\overline{P}_{ij} z_{ij} \le P_{ij} \le \overline{P}_{ij} z_{ij},\quad \sum_j P_{ji} - \sum_j P_{ij} = P_i^d s_i \quad (i \neq 0).$$

$$\sum_j f_{ji} - \sum_j f_{ij} = y_i \quad (i \neq 0),\quad |f_{ij}| \le 5 z_{ij},\quad \sum_{(i,j)\in E} z_{ij} = \sum_{i\in N} y_i - 1.$$

N, E, i, j	Fixture's six buses and candidate edges, with source bus 0; each edge has one arbitrary reference orientation. Signed flows may oppose it.
$s_i, y_i, z_{ij} \in \{0, 1\}$	Load picked up, bus energized, and branch energized. $s_i \le y_i$ permits an energized transit bus with its load disconnected.
$P_i^d \ge 0, \overline{P}_{ij} \ge 0, P_{ij} \in \mathbb{R}$	Demand, branch capacity and signed active flow, all in kW. Source export equals total picked-up demand.
$w_i > 0$	Dimensionless service weight: 10 for the two critical loads, 1 otherwise. The objective is weighted kW; finite weights are not lexicographic priority.
$f_{ij} \in \mathbb{R}$	Dimensionless connectivity commodity; each energized non-source bus consumes one unit. Source exports $\sum_{i\neq 0} y_i$; 5 bounds its total.
$y_0 = 1,\ z_{ij} \le y_i, y_j$	Source remains energized; an energized edge requires energized endpoints. Unavailable edges have $z_{ij} = 0$.

Why radial: connectivity plus the displayed edge count enforces a tree; edge count alone permits disconnected cycles. A second solve fixes weighted service and minimizes energized branch count.

Chen et al. (2018), `https://doi.org/10.1109/TPWRS.2017.2720122`; Wang et al. (2020), `https://arxiv.org/abs/1912.05185`. This is the six-node active-power fixture, not the feeder catalog.

Next: How is the residual statistic calibrated?

Optional reference: WLS, residual covariance and calibration

$$\hat{x} = \arg\min_{x \in \mathbb{R}^n} (z - Hx)^\top R^{-1} (z - Hx), \quad r = z - H\hat{x}, \quad \Omega = R - H(H^\top R^{-1} H)^{-1} H^\top .$$

$$r_i^N = |r_i| / \sqrt{\Omega_{ii}}, \qquad J = r^\top R^{-1} r, \qquad J > \chi^2_{m-n,\,1-\alpha} \implies \text{residual alarm.}$$

$x, \hat{x} \in \mathbb{R}^n$	True and estimated non-source voltage angles (radians), $n = 3$ in the four-bus DC fixture.
$z \in \mathbb{R}^m, H \in \mathbb{R}^{m \times n}$	$m = 8$ measurements and their linear map: four branch flows, three injections (pu), and one angle (rad). Each row of H maps rad to that channel's units.
$R \succ 0, \Omega \in \mathbb{R}^{m \times m}$	Noise and residual covariance matrices, with channel-dependent squared/cross units. R is diagonal for the stated independent noise.
r, r_i, r_i^N	Residual vector, its channel i component in native units, and dimensionless normalized magnitude. Defined only for $\Omega_{ii} > 0$.
$J, \chi^2_{\nu,1-\alpha}, \nu = m - n$	Dimensionless weighted residual statistic, chi-square quantile and degrees of freedom; $\alpha = .01$, $\nu = 5$, threshold 15.0863.
$\top$, R^{-1}	Matrix transpose and inverse. Implementation uses weighted least squares/linear solves; the inverse notation states the mathematics.

Calibration assumptions: correct linear model, full column rank $\operatorname{rank}(H) = n$, and Gaussian noise with the stated covariance. A residual alarm signals inconsistency; it does not identify malicious intent. Recheck rank after removing a channel.

Schweppe and Wildes (1970), `https://doi.org/10.1109/TPAS.1970.292678`; Handschin et al. (1975), `https://doi.org/10.1109/T-PAS.1975.31858`.

Next: Why can a coordinated corruption leave the residual unchanged?

Optional reference: why a coordinated corruption can be invisible

$$z' = z + Hc, \qquad \hat{x}' = \hat{x} + c, \qquad z' - H\hat{x}' = z - H\hat{x} = r.$$

Symbol / domain	Meaning in the fixed linear WLS model
$z, z' \in \mathbb{R}^m$	Original and corrupted measurement vectors, retaining each channel's native units.
$H \in \mathbb{R}^{m \times n}$	Known, unchanged full-column-rank measurement matrix; here $m = 8, n = 3$.
$c \in \mathbb{R}^n$	Attacker's selected angle shift (rad). The added measurement vector Hc has the correct units for every channel.
$\hat{x}, \hat{x}' \in \mathbb{R}^n$	Original and shifted state estimates (rad); r is their identical residual in measurement units.
$\mathcal{T}, H_{\mathcal{T}}$	Index set of protected measurement channels and the corresponding rows of H.

If protected channels cannot change, a corruption of this form must satisfy $H_{\mathcal{T}} c = 0$. When $H_{\mathcal{T}}$ has full column rank, only $c = 0$ is possible within this attack construction.

Threat assumptions: the attacker knows H and can modify every unprotected channel required by Hc. This does not establish that an arbitrary attacker can create the same effect. A low residual does not reject consistency under the stated model; it does not establish the truth of the measurements.

Liu, Ning and Reiter (2009), `https://doi.org/10.1145/1653662.1653666`. Exact fixture: `src/standard_formulations.py`, `run_wls_demo`.

Next: How does protected information enter constrained control?

Optional reference: trusted-data estimation and constrained control

$$H_{\mathcal{T}} = I_3, \quad b = FB^{-1}e_2, \qquad \min_{0 \le u \le .5} \tfrac{1}{2}u^2 \quad \text{s.t.} \quad |F\hat{x}_{\mathcal{T}} + bu| \le \overline{f} - .01\mathbf{1}.$$

$\mathcal{T}, H_{\mathcal{T}}, I_3$	Three assumed trustworthy angle channels, their measurement matrix, and the 3×3 identity. Additional independent angle channels make the toy state observable; this identity is not a row subset of the original eight-channel matrix.
$\hat{x}_{\mathcal{T}} \in \mathbb{R}^3$	Estimate from protected angles, in rad. Three measurements minus three states leave zero residual-test degrees of freedom.
$u \in [0, .5]$	Flexible active injection at bus 2, in pu, balanced by the source. The dimensionless quadratic cost selects the smallest feasible injection.
$F \in \mathbb{R}^{4\times 3}, B \in \mathbb{R}^{3\times 3}$	DC maps from non-source angles to branch flows and nodal injections; entries convert rad to pu power.
$e_2 = (0, 1, 0)^\top,\ b \in \mathbb{R}^4$	Unit selector of bus-2 injection and induced branch-flow change per unit injection.
$\overline{f} = (.35, .18, .35, .35)^\top$	Absolute branch-flow limits in pu for branches 01, 12, 23, 03. $\mathbf{1} \in \mathbb{R}^4$ is an all-ones vector.
$F\hat{x}_{\mathcal{T}} + bu$, .01	Predicted branch flows and declared planning margin, both in pu; absolute value and inequality act per branch.

Check after solving: verify finite values, u bounds and predicted constraints; then evaluate the action against the known synthetic true state. Protected-data control removes the toy overload, subject to the assumed channel trust. The margin is not a calibrated uncertainty bound.

Estimation/attack context: `https://doi.org/10.1109/TPAS.1970.292678`; `https://doi.org/10.1145/1653662.1653666`. Constrained DC teaching response: `src/standard_formulations.py`.

Next: Which assumptions still limit the scientific claim?

Study boundaries and the next scientific comparisons

Record	What it measures
Proposal-complete suite	Six settings × two supervisors; 24 Llama calls, plus one separate four-call Qwen example. One disclosed development revision; five cost ties and one worse Llama outcome.
Initial workshop runs	Continuous SAC/MPC proposals, checks and local Llama fallback choices under both model backends.
Language confirmation	12 authored requests; 8 clear and 4 requiring clarification, with independent semantic scoring and the same downstream tools.
SAC training	12,000 nominal-domain interactions; six authored model/case evaluations. One training seed does not quantify algorithm variability.
Standard-method traces	One permitted numerical attempt around continuous P/Q, a six-node MILP or four-bus DC WLS recovery.
Historical 32-run corpus	Eight finite-catalog feeder fixtures, two model backends, two supervisors and separate AC reference audits.

Language confirmation: correct completion 5/8 for the parser and 7/8 for selective Llama; wrong READY interpretations 1/12 and 2/12. Numerators count observed outcomes; denominators are clear requests (8) or all requests (12). The restoration subset does not replace the parent-study result.

Next comparison: predeclare unseen cases, repeat model calls and training seeds, match tool/retry budgets, and report incorrect recommendations separately from correct abstentions. Human time and workload need a separate human study.

Study design: `appendix/Agentic_Advantage_Study.md`. Recent benchmark context: Mansutti et al. (2026 preprint), `https://arxiv.org/abs/2609.00384`. Reading map: `appendix/Recent_Agentic_AI_Literature.md`.

Next: Choose one matched comparison to investigate after class.